TreemapMix: Dirichlet-Controlled Multi-Image Augmentation for Probability and Ordinal Supervision

Konstantin Garbers[1]*
konstantin.garbers25@stu.pku.edu.cn

Ejafa Bassam*†
ejafabassam@alumni.pku.edu.cn

Yingsheng Geng[2]
gys@bupt.edu.cn

Dalton Jens[1]
daltonjens@gmail.com

Jiaxu Qian[1]
jiaxu@pku.edu.cn

Guyue Liu[1]‡
guyue@pku.edu.cn

[1]Peking University
[2]Beijing University of Posts and Telecommunications

Abstract

Data augmentation improves visual recognition by exposing models to synthetic training examples that encourage generalization beyond the observed data. Mixed-sample methods extend this idea by combining multiple images. However, many common approaches either mix only pairs of images or rely on fixed small layouts, limiting control over source count, visible area proportions, and induced supervision. We propose TreemapMix, a distribution-controlled multi-image augmentation that samples source weights from a Dirichlet distribution and assigns them to area-proportional regions using a treemap partition. The resulting known region areas support two complementary objectives. TreemapMix with soft cross-entropy (SCE) treats areas as soft class-probability targets, while TreemapMix with a Plackett-Luce (PL) objective converts areas into ordinal supervision. On ImageNet-1K under matched training recipes, TreemapMix-SCE achieves substantially lower calibration error than evaluated mixed-sample baselines, while TreemapMix-PL achieves the strongest Top-1 accuracy. Transfer experiments on object detection and instance segmentation suggest that TreemapMix-pretrained features remain useful beyond classification. These results show that controlled multi-image composition can expose the same region-area information as either probability-matching or rank-matching supervision. Code is available at github.com/Ejafa/treemapmix.

1 Introduction

Image classification is usually formulated as learning one target label from one image. Natural visual experience is more compositional: multiple objects often appear in the same field of view, and visual categories are often understood through comparisons among related sources of evidence. This suggests a complementary augmentation principle: construct training examples that expose several labeled sources jointly, while preserving explicit control over how much visual evidence each source contributes.

Mixed-sample data augmentation moves in this direction by composing examples and mixing their labels. Mixup [48] and CutMix [47] are widely used in modern image-classification recipes [25, 40, 41, 44], but they are primarily pairwise: Mixup globally interpolates two images, and CutMix

*Equal contribution.
†This work was conducted while Ejafa Bassam was at Peking University.
‡Corresponding author.

40th Conference on Neural Information Processing Systems (NeurIPS 2026).



(a) None (b) Mixup (c) CutMix (d) RICAP (e) TreemapMix

Figure 1: Qualitative comparison of five augmentation strategies on matched source images using resize-and-center-crop preprocessing only. Mixup blends full images, CutMix pastes a random region, RICAP composes four tiles, and TreemapMix composes K rescaled source images within a randomized layout.

replaces one region with a patch from another image. Multi-image methods such as RICAP [39] and Mosaic-style augmentation [3] show the value of composing several images, but typically use a fixed small number of sources and restricted layout families. Moreover, existing methods usually treat region area only as a soft-label weight, rather than as a controllable object that can shape both the visual composition and the supervision signal. Figure 1 shows a qualitative comparison of these augmentation patterns on matched source images.

We propose *TreemapMix*, a K-way image-mixing augmentation that composes multiple single-label images into a shared canvas while preserving the training sample count. TreemapMix samples region weights from a Dirichlet distribution and maps them to non-overlapping area-proportional regions using a squarified treemap layout [4]. The Dirichlet concentration parameter α controls the composition, ranging from dominant-region mixtures to more balanced multi-image mixtures. To prevent persistent label-to-area or label-to-rank bias, TreemapMix generates K cyclic views from each group of K source images, so that every source image is paired with every sampled region weight across the cyclic orbit. Area-preserving symmetries then increase spatial diversity without changing the target weights.

TreemapMix also exposes a supervision choice. Region area provides a natural but imperfect proxy for visual evidence: larger regions usually contain more pixels, more context, and more object parts, and are therefore often easier to recognize than smaller regions. This suggests an ordinal learning signal within a composed image, where labels assigned to larger regions should generally receive higher scores than labels assigned to smaller regions. However, area is not a calibrated measure of semantic evidence; a large region may contain mostly background, while a small region may contain a discriminative object part. We therefore study two complementary objectives: TreemapMix-SCE, which treats region areas as soft probability targets and optimizes soft cross-entropy, and TreemapMix-PL, which converts region areas into area-induced ordinal supervision by ranking the target classes and optimizing a Plackett-Luce listwise objective [31, 34, 45].

We evaluate TreemapMix on ImageNet-1K [10] under matched training recipes across multiple modern image-classification backbones. Compared with 13 baseline configurations, TreemapMix-PL achieves the strongest Top-1 accuracy across the evaluated ImageNet-1K backbones, while TreemapMix-SCE gives the lowest expected calibration error (ECE). The original NLL comparison also favors SCE, while Brier-score gains are smaller and mixed (Appendix F.1). Further diagnostics show that TreemapMix-PL improves alignment between logits and the area-induced ordering of composed regions, and COCO transfer experiments indicate that TreemapMix-pretrained representations remain useful for detection and instance segmentation [27].

Our contributions are:

i. We introduce TreemapMix, a Dirichlet-controlled K-way image-mixing augmentation that supports variable-source composition with explicit area-proportional treemap layouts while preserving the training sample count.

ii. We formulate two supervision objectives from the same composed image: area-proportional soft cross-entropy for probability matching and Plackett-Luce ranking for area-induced ordinal supervision.

iii. We analyze how the Dirichlet concentration parameter controls target variability, probability-matching supervision, and ranking emphasis.

iv. We provide an empirical evaluation on ImageNet-1K, including accuracy, calibration metrics, area-logit alignment analysis, and COCO detection and instance-segmentation transfer experiments. We also evaluate long-horizon training, loader throughput and host memory, long-tailed recognition, and additional model architectures.

2 Related Work

Standard augmentations preserve the original label, including region dropout methods such as Cutout and Random Erasing [11, 49] and policy-based methods such as AutoAugment, RandAugment, and AugMix [7, 8, 16]. Pairwise mixed-sample methods instead combine two examples and assign mixed targets: Mixup [48] interpolates images and labels, while CutMix [47] uses spatial replacement with area-based labels. Later methods refine pairwise composition or label estimation through resized patches, Fourier masks, grids, saliency, class activation maps, attention, or token-level mixing [1, 5, 14, 17, 22, 28, 35]. These methods improve label-preserving or two-source augmentation, whereas TreemapMix constructs multi-source examples with a controllable distribution over two or more visible labels.

Multi-image augmentation and label supervision: Multi-image augmentations compose more than two sources, including RICAP, which patches four crops with area-weighted labels [39], Mosaic augmentation in detection pipelines [3], Cut-thumbnail [46], StackMix [6], and DCutMix [19]. Other methods generalize mixing beyond spatial layouts, such as k-Mixup through optimal-transport interpolation [13] and MixMo through a multi-input multi-output architecture [36]. TreemapMix is closest to spatial multi-image methods such as RICAP and Mosaic, but samples a single coherent K-way area distribution from a Dirichlet distribution and realizes it as an area-proportional treemap layout; unlike k-Mixup and MixMo, it remains spatially explicit and uses a standard single-input model interface. Most spatial mixing methods treat visible area as a soft-label weight, although area may be an imperfect proxy for semantic evidence. Related work studies confidence and target behavior under mixup-style training through SmoothMix for randomized smoothing [18], RegMixup [33], and RankMixup [32]. Plackett-Luce ranking has also been used for classification in Plackett–Luce Distillation (PLD) [2], but in a knowledge-distillation setting where a teacher model provides the class ranking. In contrast, TreemapMix induces the ranking directly from the mixed input: the Dirichlet-sampled region areas define the area-induced ordering over target classes. TreemapMix-SCE follows the area-weighted probability interpretation, while TreemapMix-PL converts the same areas into ordinal supervision with a Plackett-Luce ranking objective.

3 Method

TreemapMix is a K-way image-mixing augmentation that replaces each group of K source images with K cyclic mixed samples. Each mixed sample is supervised either by a soft probability-matching objective or by a ranking-based Plackett–Luce objective. We use integer $K \geq 1$ for the group size, C for the number of classes, and $\mathcal{G} = \{(x_k, y_k)\}_{k=1}^{K}$, where $y_k \in \{1, \ldots, C\}$, for a source group.

3.1 TreemapMix Augmentation

For each group, sample $w \sim \mathrm{Dirichlet}(\alpha, \ldots, \alpha)$, $\alpha > 0$, with $\sum_k w_k = 1$. These are slot weights, not fixed source-image weights. Smaller α favors dominant-region mixtures; larger α yields more balanced mixtures.

For the balanced-data protocol, TreemapMix first forms class-balanced rows from the training set and then splits each row into groups of size K. This encourages diverse multi-class mixtures and reduces the tendency of frequent classes to dominate mixed samples. When $K \leq C$, complete groups formed from a class-balanced row contain distinct labels by construction.

For stable layout construction and for ranking-based supervision, TreemapMix sorts the sampled weights in ascending order: $\lambda_1 \leq \lambda_2 \leq \cdots \leq \lambda_K, \lambda_i = w_{\tau(i)}$ (1). Here τ is a permutation of the source indices. The sorted weights define K layout slots. Slot i has area weight λ_i, with slot 1 the

smallest-area slot and slot K the largest-area slot. Given the sorted weights, TreemapMix partitions the image canvas $\Omega = [0, W) \times [0, H)$ into non-overlapping rectangular boxes $\mathcal{B} = \{B_1, \ldots, B_K\}$ with $|B_i|/(HW) \approx \lambda_i$, such that $B_i \cap B_j = \emptyset$ for $i \neq j$ and $\bigcup_{i=1}^{K} B_i = \Omega$.

Each box includes its left/top boundaries and excludes its right/bottom boundaries, so adjacent boxes share no pixels. We use a squarified treemap layout [4] to obtain area-proportional regions while favoring compact rectangles. This helps preserve useful semantic content inside each patch. The normalized treemap boxes are converted to integer pixel coordinates. If integer conversion yields zero-area boxes, TreemapMix falls back to a stripe layout with approximate area matching; target weights remain unchanged.

Area-preserving spatial symmetries increase diversity without changing targets: the eight D_4 symmetries for square images, or identity, 180° rotation, and horizontal/vertical flips for rectangular images.

TreemapMix then generates K cyclic views from each group. Let slot i have weight λ_i, box B_i, and binary mask M_i. For cyclic view $s \in \{0, \ldots, K-1\}$, define $\pi_s(i) = 1 + \big((i + s - 1) \bmod K\big)$ (2). The s-th mixed image is $\tilde{x}^{(s)} = \sum_{i=1}^{K} M_i \odot \mathrm{Resize}(x_{\pi_s(i)}, B_i)$, with sparse label-weight target $\mathcal{T}^{(s)} = \{(y_{\pi_s(i)}, \lambda_i)\}_{i=1}^{K}$ (3). Here $\odot$ denotes elementwise multiplication, and Resize embeds the resized patch in the full canvas. Thus, in each cyclic view, the source image assigned to slot i receives the slot weight λ_i as its target weight.

All K cyclic views are retained for complete groups, avoiding enumeration of $K!$ assignments. Each source visits each slot once, so mean SCE target mass is m_c/K for source-label multiplicity m_c, uniform for distinct labels (Lemma 3). This prevents persistent source-to-slot bias, but does not remove prediction-dependent PL competition. Partial orbits lack the full balancing guarantee.

Appendix A gives pseudocode, remainder handling, geometry, and fallback details. Output count matches the constructed row stream. The sparse target supports soft probability supervision, or PL ranking for distinct-label groups.

3.2 Soft TreemapMix Cross-Entropy

Let $z = f_\theta(\tilde{x})$ be the model logits (class scores before softmax), with $p_c = \mathrm{softmax}(z)_c$. We write $\mathbf{1}[\cdot]$ for the indicator function. For a TreemapMix sample $\tilde{x}$ with sparse target $\mathcal{T} = \{(c_i, \eta_i)\}_{i=1}^{K}$, where $\eta_i \geq 0$ and $\sum_{i=1}^{K} \eta_i = 1$, and where c_i is the class assigned to slot i and η_i is its area weight, TreemapMix defines $q_c = \sum_{i=1}^{K} \eta_i \mathbf{1}[c_i = c], p_c = p_\theta(c \mid \tilde{x})$ (4). For cyclic view s, we have $c_i = y_{\pi_s(i)}$ and $\eta_i = \lambda_i$. For the distinct-label groups used in the balanced-data protocol, $q_{c_i} = \eta_i$. Thus q is a probability distribution over the C classes. We denote the cross-entropy objective of TreemapMix-SCE and the ranking objective of TreemapMix-PL by $\mathcal{L}_{\text{TM-CE}}$ and $\mathcal{L}_{\text{TM-PL}}$, respectively. We obtain the soft cross-entropy objective by minimizing the KL divergence [23] between q and p_θ. Since the entropy of q is independent of the model parameters, this is equivalent to minimizing

$$\mathcal{L}_{\text{TM-CE}} = -\sum_{c=1}^{C} q_c \log p_\theta(c \mid \tilde{x}) = -\sum_{i=1}^{K} \eta_i \log p_\theta(c_i \mid \tilde{x}). \tag{5}$$

For a mini-batch of size B, the average loss is $\mathcal{L}_{\text{TM-CE}} = -\frac{1}{B} \sum_{b=1}^{B} \sum_{i=1}^{K} \eta_{b,i} \log p_\theta(c_{b,i} \mid \tilde{x}_b)$ (6). Thus, TreemapMix-SCE treats the TreemapMix weights as area-proportional probability targets and encourages predicted probability mass to match the mixed-image area weights.

The per-sample logit gradient has the standard prediction-minus-target form: $\frac{\partial \mathcal{L}_{\text{TM-CE}}}{\partial z_c} = p_c - q_c$ (7). For the batch mean, the derivative with respect to each sample's logits additionally includes $1/B$. For complete distinct-label groups, the full cyclic orbit has exactly balanced mean targets, while a uniformly selected cyclic view retains Dirichlet variability. Appendix Lemma 1 derives its α-controlled variance under unfiltered sampling. Fixed rank positions instead follow Dirichlet order statistics. With logits fixed, the gradient $p_c - q_c$ has the same target-side variance.

3.3 TreemapMix Plackett–Luce Loss

As motivated in the introduction, area is an imperfect proxy for semantic evidence. The PL objective uses the same weights as an ordinal signal, encouraging larger-area target classes to rank above smaller-area classes rather than matching their probabilities exactly.

The PL objective assumes distinct target labels. Let $\mathcal{T} = \{(c_i, \eta_i)\}_{i=1}^{K}$ be a target with distinct c_i, and define a slot permutation σ with $r_\ell = c_{\sigma(\ell)}$ and class-associated weight $\rho_{r_\ell} = \eta_{\sigma(\ell)}$. Order these weights descending: $\rho_{r_1} \geq \rho_{r_2} \geq \cdots \geq \rho_{r_K}, r_1 \succ r_2 \succ \cdots \succ r_K$ (8). Let $\mathcal{U} = \{1, \ldots, C\} \setminus \{r_1, \ldots, r_K\}$ be the set of classes absent from the TreemapMix target, and let $p_{\mathcal{U}} = 1 - \sum_{j=1}^{K} p_{r_j}$ be their total predicted probability mass. The ranking likelihood selects classes sequentially from largest to smallest target weight, normalizing each choice over the remaining classes. Following the Plackett–Luce model [31, 34] and listwise maximum-likelihood (ListMLE) objectives [45], the descending TreemapMix-PL loss is

$$\mathcal{L}_{\text{TM-PL}} = -\sum_{\ell=1}^{K} \rho_{r_\ell} \log \frac{p_\theta(r_\ell \mid \tilde{x})}{p_{\mathcal{U}} + \sum_{j=\ell}^{K} p_\theta(r_j \mid \tilde{x})} \tag{9}$$

Similar to position-aware listwise objectives such as Position-Aware ListMLE [24] and PLD [2], TreemapMix-PL weights each ranking term by its target importance; this weighted surrogate is not itself a normalized ranking likelihood. TreemapMix-SCE and TreemapMix-PL therefore use the same TreemapMix target in complementary ways: TreemapMix-SCE seeks $p_\theta(c \mid \tilde{x}) \approx q_c$, whereas TreemapMix-PL seeks $r_1 \succ r_2 \succ \cdots \succ r_K$.

For implementation, we use the ascending order. Let $a_i = r_{K+1-i}$, $\gamma_i = \rho_{a_i}$, and $\gamma_1 \leq \gamma_2 \leq \cdots \leq \gamma_K$. Thus, a_1 is the weakest ranked target class and a_K is the strongest. Define $D_i = p_{\mathcal{U}} + \sum_{j=1}^{i} p_{a_j} = 1 - \sum_{j=i+1}^{K} p_{a_j}$ (10). Then the loss becomes $\mathcal{L}_{\text{TM-PL}} = \sum_{i=1}^{K} \gamma_i \left[\log D_i - \log p_{a_i}\right]$ (11). If $T_i = \{a_{i+1}, \ldots, a_K\}$ (12) denotes the set of classes stronger than a_i, then the logit gradient is

$$\frac{\partial \mathcal{L}_{\text{TM-PL}}}{\partial z_c} = \sum_{i=1}^{K} \gamma_i \left[\frac{p_c\big(1 - \mathbf{1}[c \in T_i]\big)}{D_i} - \mathbf{1}[c = a_i] \right] \tag{13}$$

At the logit level, this gradient suppresses unranked classes, promotes the strongest ranked class, and regulates intermediate ranked classes through competition with stronger classes. The full likelihood derivation, ascending reindexing, and gradient proof are given in Appendix C.

For distinct target labels, the ascending PL weights satisfy $\gamma_i = \lambda_i$. The sorted Dirichlet weights determine the importance assigned to each rank position. Appendix Lemma 2 in Appendix C characterizes how α controls this ranking emphasis through the order statistics of the sampled Dirichlet weights. This analysis holds logits and rank assignments fixed and assumes unfiltered Dirichlet sampling; filtering changes the order-statistic moments. As α increases, the sorted weights concentrate near $1/K$, so in the distinct-label case the loss approaches $1/K$ times an unweighted top-K ListMLE-style objective. Equal-weight ties retain source-slot order.

For completeness, we also evaluate a convex combination of TreemapMix-SCE and TreemapMix-PL. Let $\beta \in [0, 1]$ denote the interpolation coefficient, where $\beta = 1$ recovers TreemapMix-SCE and $\beta = 0$ recovers TreemapMix-PL. We define $\mathcal{L}_{\text{TM-Mix}} = \beta \mathcal{L}_{\text{TM-CE}} + (1 - \beta)\mathcal{L}_{\text{TM-PL}}$ (14). Using the ascending PL notation from Section 3.3, this can be written as $\mathcal{L}_{\text{TM-Mix}} = \sum_{i=1}^{K} \gamma_i \left[(1 - \beta) \log D_i - \log p_{a_i}\right]$ (15). This objective is not intended as a separate contribution; we include it to test whether combining probability matching and rank matching improves the accuracy–calibration trade-off.

3.4 Dirichlet-Controlled Supervision

TreemapMix supports a fixed Dirichlet concentration or a schedule over training progress. We vary α between endpoints $\alpha_{\min}$ and $\alpha_{\max}$ using a linear or cosine schedule. A forward schedule increases α, shifting from dominant-region mixtures toward more balanced mixtures; a backward schedule decreases it.

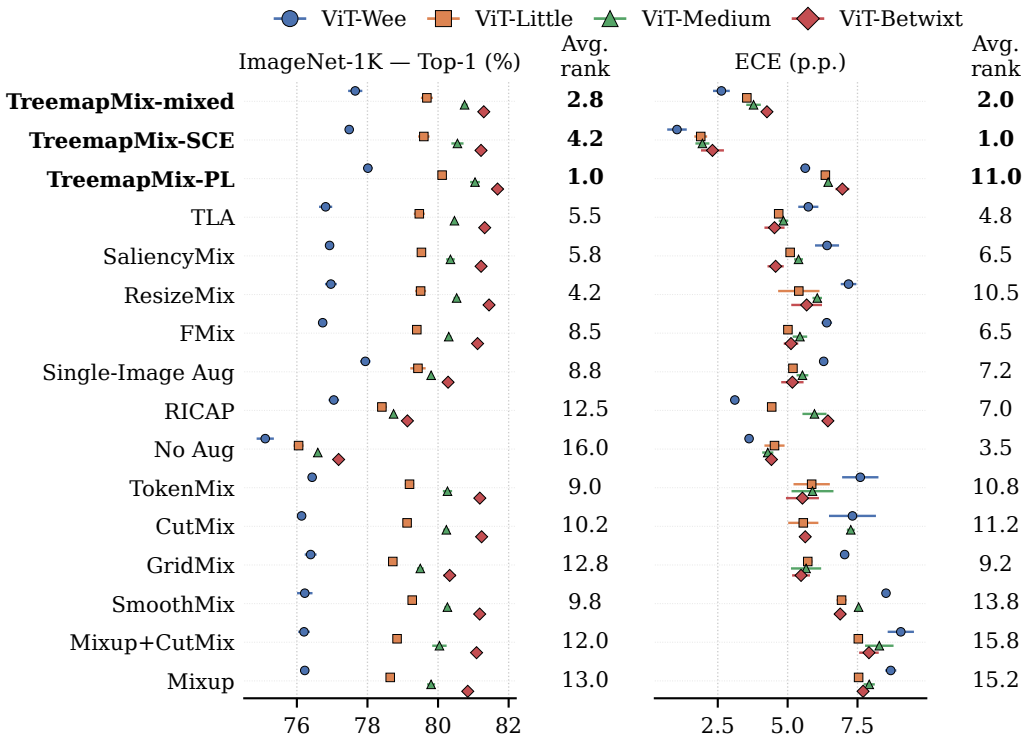


Figure 2: ImageNet-1K accuracy and calibration for all 16 configurations under the matched 110-epoch protocol. Points show means over three seeds; horizontal intervals denote ± 1 sample standard deviation. Marker shape and color identify the backbone. ECE uses 15 bins and is reported in percentage points. Average ranks are computed within each backbone (lower is better).

4 Empirical Evaluation

Protocol. Our main ImageNet-1K study uses four compact ViT/EVA02-family backbones in `timm` [12], two NVIDIA H20 GPUs per run, global batch size 1024, and a matched 110-epoch recipe. Unless specified otherwise, results are mean $\pm$ one sample standard deviation over seeds 42, 43, and 44. Accuracy is Top-1 percent; ECE is the sample-weighted absolute accuracy–confidence gap across 15 equal-width bins, in percentage points. TreemapMix uses $K = 4$ for SCE/mixed, $K = 6$ for PL, and cosine schedule $\alpha : 0.1 \to 0.5$; mixed uses $\beta = 0.5$. Hyperparameter settings selected on ViT-Wee are fixed across backbones. Appendix E.1 gives compute, recipes, and hyperparameters; Appendix E.3 gives mathematical loss references.

ImageNet-1K. Tables 1 and 2 report all 16 configurations, including the additional review-time baselines. FMix, GridMix, ResizeMix, SaliencyMix, SmoothMix, TokenMix, and TLA are OpenMixup-based reimplementations [25], each selected from eight configurations on ViT-Wee. PL has the highest mean accuracy on every backbone, whereas SCE has the lowest ECE. The strongest baseline varies: single-image augmentation on Wee, SaliencyMix on Little, and ResizeMix on Medium/Betwixt. The mixed loss does not consistently improve on both objectives. Figure 2 compares Top-1 accuracy and ECE for all 16 configurations; NLL/Brier results appear in Appendix F.1.

Other datasets and architectures. Appendix Tables 15 and 16 contain the complete CIFAR-100 comparison at 256-pixel resolution under 210 epochs including 10 warmup epochs. SCE has the lowest ECE on all backbones; PL has the highest accuracy on three, while ResizeMix leads on Little. Under the main ImageNet-1K 110-epoch protocol and TreemapMix settings, Table 3 retains the accuracy/calibration separation on ConvNeXtV2-Nano, DeiT-Tiny, and Swin-Tiny. Small PL–SCE accuracy gaps on DeiT/Swin are not claimed statistically significant.

Table 1: ImageNet-1K Top-1 accuracy (%) under the matched 110-epoch protocol. Mean ± sample standard deviation over three seeds; bold marks the best mean per backbone.

Method	ViT-Wee ↑	ViT-Little ↑	ViT-Medium ↑	ViT-Betwixt ↑
No Aug	75.10 ± 0.24	76.05 ± 0.14	76.59 ± 0.02	77.18 ± 0.14
Single-Image Aug	77.94 ± 0.15	79.43 ± 0.22	79.80 ± 0.04	80.28 ± 0.16
Mixup+CutMix	76.20 ± 0.16	78.84 ± 0.05	80.04 ± 0.20	81.09 ± 0.13
Mixup	76.22 ± 0.12	78.64 ± 0.10	79.80 ± 0.11	80.84 ± 0.09
CutMix	76.13 ± 0.07	79.12 ± 0.07	80.23 ± 0.09	81.23 ± 0.12
RICAP	77.05 ± 0.15	78.41 ± 0.14	78.74 ± 0.07	79.13 ± 0.11
FMix	76.73 ± 0.10	79.40 ± 0.06	80.30 ± 0.07	81.12 ± 0.10
GridMix	76.39 ± 0.17	78.72 ± 0.05	79.50 ± 0.09	80.33 ± 0.18
ResizeMix	76.96 ± 0.17	79.51 ± 0.15	80.53 ± 0.09	81.44 ± 0.13
SaliencyMix	76.93 ± 0.11	79.53 ± 0.07	80.35 ± 0.12	81.22 ± 0.06
SmoothMix	76.22 ± 0.22	79.27 ± 0.09	80.26 ± 0.08	81.18 ± 0.05
TokenMix	76.43 ± 0.10	79.19 ± 0.13	80.27 ± 0.13	81.18 ± 0.10
TLA	76.81 ± 0.19	79.47 ± 0.16	80.46 ± 0.03	81.32 ± 0.13
TreemapMix-SCE	77.48 ± 0.03	79.60 ± 0.16	80.55 ± 0.17	81.22 ± 0.15
TreemapMix-mixed	77.65 ± 0.20	79.68 ± 0.17	80.75 ± 0.09	81.30 ± 0.05
TreemapMix-PL	$\mathbf{78.01 \pm 0.13}$	$\mathbf{80.12 \pm 0.04}$	$\mathbf{81.05 \pm 0.14}$	$\mathbf{81.69 \pm 0.16}$

Table 2: ImageNet-1K ECE (15 bins, percentage points). Same runs as Table 1; bold marks the lowest mean per backbone.

Method	ViT-Wee ↓	ViT-Little ↓	ViT-Medium ↓	ViT-Betwixt ↓
No Aug	3.62 ± 0.13	4.53 ± 0.36	4.29 ± 0.20	4.42 ± 0.02
Single-Image Aug	6.29 ± 0.13	5.19 ± 0.12	5.53 ± 0.21	5.17 ± 0.40
Mixup+CutMix	9.05 ± 0.47	7.53 ± 0.10	8.28 ± 0.51	7.91 ± 0.35
Mixup	8.69 ± 0.20	7.54 ± 0.15	7.92 ± 0.20	7.70 ± 0.11
CutMix	7.32 ± 0.84	5.56 ± 0.54	7.26 ± 0.14	5.63 ± 0.20
RICAP	3.11 ± 0.02	4.43 ± 0.11	5.96 ± 0.43	6.44 ± 0.11
FMix	6.40 ± 0.19	5.01 ± 0.10	5.44 ± 0.26	5.12 ± 0.26
GridMix	7.04 ± 0.15	5.72 ± 0.17	5.66 ± 0.54	5.48 ± 0.32
ResizeMix	7.18 ± 0.28	5.40 ± 0.74	6.06 ± 0.17	5.68 ± 0.55
SaliencyMix	6.41 ± 0.43	5.09 ± 0.09	5.39 ± 0.16	4.57 ± 0.29
SmoothMix	8.52 ± 0.06	6.93 ± 0.18	7.54 ± 0.06	6.88 ± 0.21
TokenMix	7.60 ± 0.65	5.86 ± 0.65	5.89 ± 0.75	5.53 ± 0.59
TLA	5.74 ± 0.36	4.68 ± 0.03	4.85 ± 0.17	4.53 ± 0.36
TreemapMix-SCE	$\mathbf{1.04 \pm 0.35}$	$\mathbf{1.89 \pm 0.23}$	$\mathbf{1.95 \pm 0.25}$	$\mathbf{2.31 \pm 0.41}$
TreemapMix-mixed	2.63 ± 0.30	3.54 ± 0.05	3.78 ± 0.26	4.26 ± 0.13
TreemapMix-PL	5.63 ± 0.18	6.35 ± 0.10	6.45 ± 0.05	6.96 ± 0.25

Table 3: Cross-architecture ImageNet-1K results under 110 epochs. Mean ± sample standard deviation over three seeds; accuracy in percent and ECE in percentage points (15 bins). Bold marks the best mean per metric.

	ConvNeXtV2-Nano		DeiT-Tiny		Swin-Tiny	
Method	Acc. ↑	ECE ↓	Acc. ↑	ECE ↓	Acc. ↑	ECE ↓
Mixup+CutMix	79.79 ± 0.11	9.48 ± 0.13	68.86 ± 0.28	12.34 ± 0.27	79.62 ± 0.06	7.50 ± 0.13
RICAP	79.08 ± 0.10	5.95 ± 0.17	71.71 ± 0.04	2.04 ± 0.28	78.68 ± 0.07	7.78 ± 0.05
TreemapMix-SCE	80.01 ± 0.09	$\mathbf{2.74 \pm 0.27}$	71.78 ± 0.26	$\mathbf{1.32 \pm 0.26}$	80.55 ± 0.09	$\mathbf{2.52 \pm 0.11}$
TreemapMix-PL	$\mathbf{80.45 \pm 0.10}$	6.82 ± 0.07	$\mathbf{71.81 \pm 0.07}$	2.71 ± 0.15	$\mathbf{80.57 \pm 0.12}$	6.58 ± 0.06

ImageNet-LT [29] is a long-tailed subset of ImageNet-1K, constructed by subsampling its training images to impose a power-law class-frequency distribution across the same 1,000 classes (5–1,280 images per class). For TreemapMix, the loader shuffles each class's images and forms rows by taking the next image from every class that still has one. It shuffles these rows and partitions each into K-source groups, preserving distinct labels. Incomplete groups are padded from earlier images in the same row, retaining only the required number of cyclic outputs; rows shorter than K remain unmixed. Every image occurs once in the primary schedule, preserving the original class counts without minority-class oversampling; padding reuses images while keeping the total output count unchanged. Complete groups retain cyclic source-to-slot balance. Under the matched 110-epoch protocol, using three seeds, PL improves accuracy over Mixup+CutMix by 1.40, 1.02, 1.31, and 1.82 points; SCE's calibration advantage does not persist (Table 4).

Table 4: ImageNet-LT accuracy and ECE under the matched 110-epoch protocol. Mean ± sample standard deviation over three seeds; ECE uses 15 bins. Bold marks the best mean per metric.

Method	ViT-Wee	ViT-Little	ViT-Medium	ViT-Betwixt
Top-1 accuracy (%) ↑				
Mixup+CutMix	41.72 ± 0.08	41.67 ± 0.43	40.95 ± 0.07	39.78 ± 0.21
RICAP	35.70 ± 0.15	35.39 ± 0.00	34.35 ± 0.02	31.44 ± 3.37
TreemapMix-SCE	41.44 ± 0.40	41.09 ± 0.40	39.48 ± 0.13	38.67 ± 0.27
TreemapMix-PL	$\mathbf{43.12 \pm 0.06}$	$\mathbf{42.69 \pm 0.07}$	$\mathbf{42.26 \pm 0.07}$	$\mathbf{41.60 \pm 0.17}$
ECE (percentage points) ↓				
Mixup+CutMix	$\mathbf{0.58 \pm 0.14}$	$\mathbf{0.83 \pm 0.09}$	$\mathbf{2.14 \pm 0.61}$	$\mathbf{3.31 \pm 0.96}$
RICAP	28.50 ± 0.26	31.86 ± 0.21	33.12 ± 0.32	30.89 ± 2.81
TreemapMix-SCE	10.65 ± 0.04	13.00 ± 0.56	11.85 ± 0.33	11.35 ± 0.62
TreemapMix-PL	17.90 ± 0.63	25.63 ± 0.38	23.62 ± 0.74	25.45 ± 0.79

Computational performance. Table 5 reports throughput, total loader time, and peak host memory from the review-time benchmark. Relative to single-image augmentation, SCE ($K = 4$) reduces throughput by 19.0% and PL ($K = 6$) by 25.6%, with increased host memory. These CPU-side measurements exclude model computation and do not establish end-to-end training overhead; prefetching can overlap preparation with GPU work (Appendix E.4).

Table 5: Complete loader benchmark: five runs (seeds 42–46), batch size 128, eight workers, first batch, 20 warmup and 100 measured batches. Reduction is $100(1 - T/T_{\text{single}})$; ΔRSS uses unrounded means. Memory is GiB (2^{30} bytes). Total time covers all consumed batches; throughput uses measured batches only.

Method	K	Throughput (images/s)	Throughput reduction	Total loader time (s)
No Aug	1	1670.4 ± 30.2	-30.2%	11.43 ± 0.13
Single-Image Aug	1	1282.8 ± 35.1	0.0%	14.35 ± 0.27
Mixup+CutMix	2	1223.4 ± 29.7	4.6%	15.13 ± 0.36
TreemapMix-SCE	4	1039.4 ± 37.0	19.0%	19.83 ± 0.46
TreemapMix-PL	6	954.3 ± 33.0	25.6%	21.30 ± 0.36
RICAP	4	364.8 ± 5.9	71.6%	44.00 ± 0.47

Method	Peak RSS (GiB)	ΔRSS (GiB)	Peak USS (GiB)
No Aug	28.49 ± 0.07	$+0.11$	17.82 ± 0.06
Single-Image Aug	28.38 ± 0.16	$+0.00$	17.76 ± 0.17
Mixup+CutMix	28.46 ± 0.12	$+0.08$	17.82 ± 0.15
TreemapMix-SCE	30.85 ± 0.34	$+2.47$	20.16 ± 0.28
TreemapMix-PL	32.41 ± 0.52	$+4.04$	21.74 ± 0.49
RICAP	36.82 ± 0.14	$+8.44$	25.18 ± 0.13

Sensitivity and longer training. The 320-epoch Wee/Little comparison (20 warmup epochs followed by 300 regular epochs) retains PL's highest mean accuracy and SCE's lowest ECE, with smaller PL gains over Mixup+CutMix of 0.91/0.21 points (Table 6). It reuses short-schedule settings.

Table 6: ImageNet-1K long-horizon results for ViT-Wee and ViT-Little under a 320-epoch schedule. Accuracy is reported in percent, and ECE in percentage points using 15 bins. Values are mean ± one sample standard deviation over seeds 42, 43, and 44. Bold marks the best mean for each model and metric.

	ViT-Wee		ViT-Little	
Method	Acc. ↑	ECE ↓	Acc. ↑	ECE ↓
Single-Image Aug	80.37 ± 0.08	3.89 ± 0.16	81.12 ± 0.04	3.39 ± 0.06
Mixup+CutMix	79.73 ± 0.15	6.87 ± 0.64	81.74 ± 0.09	5.47 ± 0.52
TreemapMix-SCE	80.41 ± 0.04	$\mathbf{3.23 \pm 0.49}$	81.58 ± 0.33	$\mathbf{2.77 \pm 1.42}$
TreemapMix-PL	$\mathbf{80.64 \pm 0.16}$	6.94 ± 0.14	$\mathbf{81.95 \pm 0.34}$	7.44 ± 0.54

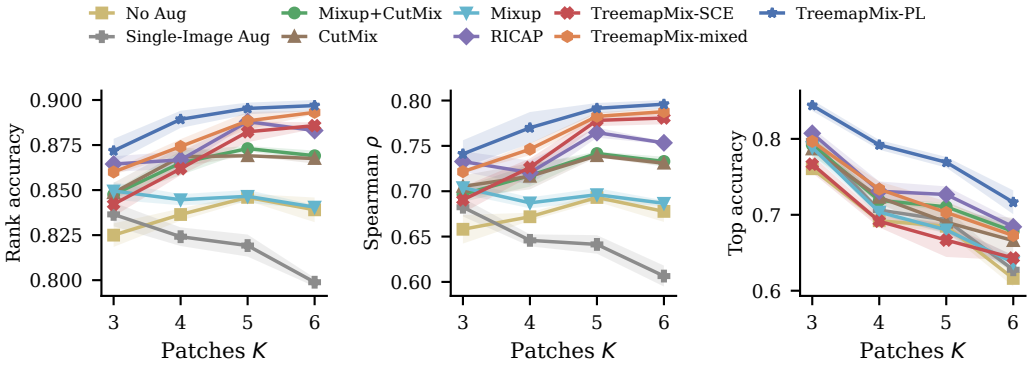


Figure 3: Area-logit alignment diagnostics on the held-out composed diagnostic set for ViT-Betwixt checkpoints from the main ImageNet-1K study. Lines show means over three seeds and shaded bands denote ± 1 sample standard deviation: pairwise rank accuracy (left), area-logit Spearman correlation (center), and largest-area top-class accuracy (right).

Single-seed optimizer and (K, α) studies appear in Appendices F.3 and F.4. The latter supports small-to-medium K and small α in the recommended regimes, without isolating filtering or scheduling effects.

5 Diagnostic and Transfer Analysis

We examine whether logits reflect the known region-area ordering and whether pretrained features transfer to COCO detection and instance segmentation. Hyperparameter sweeps and qualitative attributions appear in Appendices F.4 and F.5.

Compositional Logit Alignment: We evaluate the ViT-Betwixt checkpoints from the main ImageNet-1K comparison across three seeds on 500 held-out TreemapMix mixtures for each $K \in \{3, 4, 5, 6\}$. Figure 3 reports three diagnostics restricted to classes present in each mixture: pairwise rank accuracy is the fraction of class pairs whose logits follow their area ordering, counting only pairs whose area fractions differ by at least 0.05 (left); Spearman correlation measures agreement between area and logit ranks, excluding samples with tied areas or logits (center); and largest-area top-class accuracy is the fraction of samples whose largest-area class has the highest logit among present classes, excluding tied largest areas (right).

TreemapMix-PL gives the highest pairwise rank accuracy, area-logit Spearman correlation, and largest-area top-class accuracy in Figure 3. RICAP is competitive across all three metrics and outperforms TreemapMix-SCE on largest-area top-class accuracy at every evaluated K, despite also using area-weighted soft targets. These results suggest that the ranking objective contributes to area-logit alignment beyond the effect of compositional augmentation alone.

Table 7: COCO transfer results after ViTDet-style Mask R-CNN fine-tuning from the main ImageNet-1K experiment checkpoints with three seeds per method. Values are mean ± standard deviation over these three transferred seeds; higher is better. Box AP evaluates bounding boxes and mask AP evaluates instance masks. AP is averaged over IoU thresholds from 0.50 to 0.95 in increments of 0.05.

	ViT-Wee		ViT-Betwixt	
Method	Box AP ↑	Mask AP ↑	Box AP ↑	Mask AP ↑
Mixup+CutMix	23.66 ± 0.01	20.34 ± 0.10	29.04 ± 0.07	25.25 ± 0.10
RICAP	23.08 ± 0.17	19.82 ± 0.07	27.86 ± 0.08	24.21 ± 0.06
TreemapMix-SCE	24.00 ± 0.17	20.55 ± 0.20	29.44 ± 0.16	25.56 ± 0.14
TreemapMix-PL	$\mathbf{24.32 \pm 0.12}$	$\mathbf{20.71 \pm 0.04}$	$\mathbf{29.71 \pm 0.09}$	$\mathbf{25.75 \pm 0.10}$

Feature Transfer to Instance Segmentation: We fine-tune ViT-Wee and ViT-Betwixt ImageNet-1K checkpoints with a ViTDet-style adapter and Mask R-CNN [15, 26] on COCO for 30 epochs at 256×256 resolution. Mixup+CutMix and RICAP provide the comparison baselines. Table 7 reports box/mask AP averaged over three transferred seeds; Appendix E.2 provides fine-tuning hyperparameters and additional AP50 results (Table 12).

Both TreemapMix variants improve over Mixup+CutMix and RICAP on box AP and mask AP for both backbones, with TreemapMix-PL achieving the highest scores. These results show that TreemapMix-pretrained checkpoints transfer favorably to COCO detection and instance segmentation under this fine-tuning protocol. However, they should not be interpreted as proving that the area-logit alignment mechanism causes the downstream gains, since the transfer experiment compares pretrained checkpoints rather than isolating individual representation properties.

6 Conclusion and Limitations

We introduced TreemapMix, a Dirichlet-controlled K-way augmentation with area-proportional treemap layouts. Cyclic views preserve sample count and, for complete groups, balance each source's exposure to region sizes and ranks. The same composition supports area-weighted soft cross-entropy for probability matching and Plackett–Luce ranking for ordinal supervision. Under matched ImageNet-1K training recipes, SCE gives the strongest calibration, while PL gives the strongest Top-1 accuracy and area-logit alignment. Experiments extend this evaluation to CIFAR-100, additional architectures, long-tailed recognition, 320-epoch training, and loader efficiency. COCO transfer further demonstrates useful detection and instance-segmentation representations. Our comparisons leave the best attainable TreemapMix performance under separately optimized hyperparameters unexplored. Future work can tune recipes for longer schedules, study semantic content alongside region area, and isolate the contributions of cyclic views and class-balanced grouping. Other directions include calibration under skewed class priors, end-to-end training efficiency, adaptive grouping and schedules for K and α, and extensions to contrastive or self-supervised learning.

Acknowledgments and Disclosure of Funding

This work is supported by the New Generation Artificial Intelligence–National Science and Technology Major Project (No. 2026ZD0124304) and the National Natural Science Foundation of China (NSFC) under Grant No. 92582109. We also thank Tencent for providing GPU computing resources.

The authors declare no competing interests.

References

[1] Kyungjune Baek, Duhyeon Bang, and Hyunjung Shim. GridMix: Strong regularization through local context mapping. 109:107594, 2021. ISSN 0031-3203. doi: 10.1016/j.patcog.2020.107594. URL https://www.sciencedirect.com/science/article/pii/S0031320320303976.

[2] Ejafa Bassam, Dawei Zhu, and Kaigui Bian. Pld: A choice-theoretic list-wise knowledge distillation. *Advances in Neural Information Processing Systems*, 38:136090–136112, 2026.

[3] Alexey Bochkovskiy, Chien-Yao Wang, and Hong-Yuan Mark Liao. YOLOv4: Optimal Speed and Accuracy of Object Detection, 2020. URL http://arxiv.org/abs/2004.10934.

[4] Mark Bruls, Kees Huizing, and Jarke J. van Wijk. Squarified treemaps. In *Data Visualization 2000*, pages 33–42, Vienna, 2000. Springer. doi: 10.1007/978-3-7091-6783-0_4.

[5] Jie-Neng Chen, Shuyang Sun, Ju He, Philip HS Torr, Alan Yuille, and Song Bai. Transmix: Attend to mix for vision transformers. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 12135–12144, 2022.

[6] John Chen, Samarth Sinha, and Anastasios Kyrillidis. Stackmix: a complementary mix algorithm. In James Cussens and Kun Zhang, editors, *Proceedings of the Thirty-Eighth Conference on Uncertainty in Artificial Intelligence*, volume 180 of *Proceedings of Machine Learning Research*, pages 326–335. PMLR, 01–05 Aug 2022. URL https://proceedings.mlr.press/v180/chen22b.html.

[7] Ekin D Cubuk, Barret Zoph, Dandelion Mane, Vijay Vasudevan, and Quoc V Le. Autoaugment: Learning augmentation strategies from data. In *2019 IEEE/CVF conference on computer vision and pattern recognition (CVPR)*, pages 113–123. IEEE, 2019.

[8] Ekin Dogus Cubuk, Barret Zoph, Jon Shlens, and Quoc Le. Randaugment: Practical automated data augmentation with a reduced search space. *Advances in neural information processing systems*, 33:18613–18624, 2020.

[9] George E. Dahl, Frank Schneider, Zachary Nado, Naman Agarwal, Chandramouli Shama Sastry, Philipp Hennig, Sourabh Medapati, Runa Eschenhagen, Priya Kasimbeg, Daniel Suo, Juhan Bae, Justin Gilmer, Abel L. Peirson, Bilal Khan, Rohan Anil, Mike Rabbat, Shankar Krishnan, Daniel Snider, Ehsan Amid, Kongtao Chen, Chris J. Maddison, Rakshith Vasudev, Michal Badura, Ankush Garg, and Peter Mattson. Benchmarking Neural Network Training Algorithms, 2023.

[10] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. ImageNet: A large-scale hierarchical image database. In *2009 IEEE Conference on Computer Vision and Pattern Recognition*, pages 248–255, 2009. doi: 10.1109/CVPR.2009.5206848. URL https://ieeexplore.ieee.org/document/5206848.

[11] Terrance Devries and Graham W. Taylor. Improved regularization of convolutional neural networks with cutout. *CoRR*, abs/1708.04552, 2017. URL http://arxiv.org/abs/1708.04552.

[12] Yuxin Fang, Quan Sun, Xinggang Wang, Tiejun Huang, Xinlong Wang, and Yue Cao. Eva-02: A visual representation for neon genesis. *Image and Vision Computing*, 149:105171, 2024.

[13] Kristjan Greenewald, Anming Gu, Mikhail Yurochkin, Justin Solomon, and Edward Chien. k-mixup regularization for deep learning via optimal transport. *Transactions on Machine Learning Research*, 2023. ISSN 2835-8856. URL https://openreview.net/forum?id=lOegPKSu04.

[14] Ethan Harris, Antonia Marcu, Matthew Painter, Mahesan Niranjan, Adam Prügel-Bennett, and Jonathon Hare. Fmix: Enhancing mixed sample data augmentation, 2021. URL https://arxiv.org/abs/2002.12047.

[15] Kaiming He, Georgia Gkioxari, Piotr Dollár, and Ross Girshick. Mask r-cnn. In *Proceedings of the IEEE international conference on computer vision*, pages 2961–2969, 2017.

[16] Dan Hendrycks*, Norman Mu*, Ekin Dogus Cubuk, Barret Zoph, Justin Gilmer, and Balaji Lakshminarayanan. Augmix: A simple method to improve robustness and uncertainty under data shift. In *International Conference on Learning Representations*, 2020. URL https://openreview.net/forum?id=S1gmrxHFvB.

[17] Shaoli Huang, Xinchao Wang, and Dacheng Tao. Snapmix: Semantically proportional mixing for augmenting fine-grained data. In *Proceedings of the AAAI conference on artificial intelligence*, volume 35, pages 1628–1636, 2021.

[18] Jongheon Jeong, Sejun Park, Minkyu Kim, Heung-Chang Lee, Do-Guk Kim, and Jinwoo Shin. Smoothmix: Training confidence-calibrated smoothed classifiers for certified robustness. *Advances in Neural Information Processing Systems*, 34:30153–30168, 2021.

[19] Joonhyun Jeong, Sungmin Cha, Jongwon Choi, Sangdoo Yun, Taesup Moon, and Youngjoon Yoo. Observations on k-image expansion of image-mixing augmentation. *IEEE Access*, 11: 16631–16643, 2023. doi: 10.1109/ACCESS.2023.3243108.

[20] Keller Jordan, Yuchen Jin, Vlado Boza, You Jiacheng, Franz Cesista, Laker Newhouse, and Jeremy Bernstein. Muon: An optimizer for hidden layers in neural networks, 2024. URL https://kellerjordan.github.io/posts/muon/.

[21] Priya Kasimbeg, Frank Schneider, Runa Eschenhagen, Juhan Bae, Chandramouli Shama Sastry, Mark Saroufim, Boyuan Feng, Less Wright, Edward Yang, Zachary Nado, et al. Accelerating neural network training: An analysis of the algoperf competition. In *International Conference on Learning Representations*, volume 2025, pages 43455–43477, 2025.

[22] Jang-Hyun Kim, Wonho Choo, and Hyun Oh Song. Puzzle mix: Exploiting saliency and local statistics for optimal mixup. In *International conference on machine learning*, pages 5275–5285. PMLR, 2020.

[23] Solomon Kullback and Richard A. Leibler. On information and sufficiency. *The Annals of Mathematical Statistics*, 22(1):79–86, 1951. doi: 10.1214/aoms/1177729694.

[24] Yanyan Lan, Yadong Zhu, Jiafeng Guo, Shuzi Niu, and Xueqi Cheng. Position-aware listmle: A sequential learning process for ranking. In *Proceedings of the 30th Conference on Uncertainty in Artificial Intelligence (UAI 2014)*, pages 449–458. AUAI Press, 2014.

[25] Siyuan Li, Zedong Wang, Zicheng Liu, Juanxi Tian, Di Wu, Cheng Tan, Weiyang Jin, and Stan Z. Li. OpenMixup: Open Mixup Toolbox and Benchmark for Visual Representation Learning, 2024. URL http://arxiv.org/abs/2209.04851.

[26] Yanghao Li, Hanzi Mao, Ross Girshick, and Kaiming He. Exploring plain vision transformer backbones for object detection, 2022.

[27] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C Lawrence Zitnick. Microsoft coco: Common objects in context. In *European conference on computer vision*, pages 740–755. Springer, 2014.

[28] Jihao Liu, Boxiao Liu, Hang Zhou, Hongsheng Li, and Yu Liu. Tokenmix: Rethinking image mixing for data augmentation in vision transformers. In *European conference on computer vision*, pages 455–471. Springer, 2022.

[29] Ziwei Liu, Zhongqi Miao, Xiaohang Zhan, Jiayun Wang, Boqing Gong, and Stella X Yu. Large-scale long-tailed recognition in an open world. In *2019 IEEE/CVF conference on computer vision and pattern recognition (CVPR)*, pages 2532–2541. IEEE, 2019.

[30] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization, 2019. URL https://arxiv.org/abs/1711.05101.

[31] R. Duncan Luce. *Individual Choice Behavior: A Theoretical Analysis*. Wiley, New York, 1959.

[32] Jongyoun Noh, Hyekang Park, Junghyup Lee, and Bumsub Ham. Rankmixup: Ranking-based mixup training for network calibration. In *2023 ieee/cvf international conference on computer vision (iccv)*, pages 1358–1368. IEEE, 2023.

[33] Francesco Pinto, Harry Yang, Ser Nam Lim, Philip Torr, and Puneet Dokania. Using mixup as a regularizer can surprisingly improve accuracy & out-of-distribution robustness. *Advances in neural information processing systems*, 35:14608–14622, 2022.

[34] R. L. Plackett. The analysis of permutations. *Journal of the Royal Statistical Society. Series C (Applied Statistics)*, 24(2):193–202, 1975. doi: 10.2307/2346567.

[35] Jie Qin, Jiemin Fang, Qian Zhang, Wenyu Liu, and Xingang Wang. Resizemix: Mixing data with preserved object information and true labels, 2020. URL https://arxiv.org/abs/2012.11101.

[36] Alexandre Rame, Remy Sun, and Matthieu Cord. Mixmo: Mixing multiple inputs for multiple outputs via deep subnetworks. pages 803–813, 10 2021. doi: 10.1109/ICCV48922.2021.00086.

[37] Avanti Shrikumar, Peyton Greenside, and Anshul Kundaje. Learning important features through propagating activation differences. In *International conference on machine learning*, pages 3145–3153. PMlR, 2017.

[38] Karen Simonyan, Andrea Vedaldi, and Andrew Zisserman. Deep Inside Convolutional Networks: Visualising Image Classification Models and Saliency Maps, 2014. URL http://arxiv.org/abs/1312.6034.

[39] Ryo Takahashi, Takashi Matsubara, and Kuniaki Uehara. Data Augmentation using Random Image Cropping and Patching for Deep CNNs. *IEEE Transactions on Circuits and Systems for Video Technology*, 30(9):2917–2931, 2020. ISSN 1051-8215, 1558-2205. doi: 10.1109/TCSVT.2019.2935128. URL http://arxiv.org/abs/1811.09030.

[40] Hugo Touvron, Matthieu Cord, Matthijs Douze, Francisco Massa, Alexandre Sablayrolles, and Herve Jegou. Training data-efficient image transformers & distillation through attention. In Marina Meila and Tong Zhang, editors, *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 10347–10357. PMLR, 18–24 Jul 2021. URL https://proceedings.mlr.press/v139/touvron21a.html.

[41] Hugo Touvron, Matthieu Cord, and Hervé Jégou. Deit iii: Revenge of the vit. In *European conference on computer vision*, pages 516–533. Springer, 2022.

[42] Ross Wightman. Pytorch image models, 2019.

[43] Ross Wightman. Searching for Better Vit Baselines Hparams, 2022. URL https://gist.github.com/rwightman/fb37c339efd2334177ff99a8083ebbc4.

[44] Ross Wightman, Hugo Touvron, and Hervé Jégou. ResNet strikes back: An improved training procedure in timm, 2021. URL http://arxiv.org/abs/2110.00476.

[45] Fen Xia, Tie-Yan Liu, Jue Wang, Wensheng Zhang, and Hang Li. Listwise approach to learning to rank: Theory and algorithm. In *Proceedings of the 25th International Conference on Machine Learning*, ICML '08, pages 1192–1199, New York, NY, USA, 2008. Association for Computing Machinery. doi: 10.1145/1390156.1390306.

[46] Tianshu Xie, Xuan Cheng, Xiaomin Wang, Minghui Liu, Jiali Deng, Tao Zhou, and Ming Liu. Cut-thumbnail: A novel data augmentation for convolutional neural network. In *Proceedings of the 29th ACM International Conference on Multimedia*, MM '21, page 1627–1635. ACM, October 2021. doi: 10.1145/3474085.3475302. URL http://dx.doi.org/10.1145/3474085.3475302.

[47] Sangdoo Yun, Dongyoon Han, Seong Joon Oh, Sanghyuk Chun, Junsuk Choe, and Youngjoon Yoo. CutMix: Regularization strategy to train strong classifiers with localizable features. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 6023–6032, 2019. doi: 10.1109/ICCV.2019.00612. URL https://openaccess.thecvf.com/content_ICCV_2019/html/Yun_CutMix_Regularization_Strategy_to_Train_Strong_Classifiers_With_Localizable_Features_ICCV_2019_paper.html.

[48] H Zhang, M Cissé, YN Dauphin, and D Lopez-Paz. mixup: beyond empirical risk minimization. iclr. *Vancouver, BC, Canada, April*, 2018. URL https://openreview.net/forum?id=r1Ddp1-Rb. Poster.

[49] Zhun Zhong, Liang Zheng, Guoliang Kang, Shaozi Li, and Yi Yang. Random erasing data augmentation. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, 2020.

A TreemapMix Augmentation Details

This appendix provides implementation details omitted from the main Method section. We use the notation introduced in Section 3.

Algorithm 1 TreemapMix augmentation

Require: dataset $\mathcal{D}$, group size K, concentration $\alpha > 0$, image size $H \times W$, with $1 \le K \le \min(C, W)$ for this balanced-row protocol

build class-balanced rows and split them into groups of size K
for each group $\mathcal{G} = \{(x_k, y_k)\}_{k=1}^{K}$ **do**
  sample $w \sim \mathrm{Dirichlet}(\alpha, \dots, \alpha)$
  sort weights into $\lambda_1 \le \cdots \le \lambda_K$
  construct boxes $B_1, \dots, B_K$ with $|B_i|/(HW) \approx \lambda_i$
  apply a random valid symmetry transform to the boxes
  for $s = 0, \dots, K-1$ **do**
    assign each source image $x_{\pi_s(i)}$ to slot i, for $i = 1, \dots, K$
    render $\tilde{x}^{(s)} = \sum_i M_i \odot \mathrm{Resize}(x_{\pi_s(i)}, B_i)$
    output $\big(\tilde{x}^{(s)}, \{(y_{\pi_s(i)}, \lambda_i)\}_{i=1}^{K}\big)$
  end for
end for

A.1 Class-balanced row construction

Let the training dataset be $\mathcal{D} = \{(x_n, y_n)\}_{n=1}^{N_{\mathrm{data}}}$, where $y_n \in \{1, \dots, C\}$, and let $\mathcal{I}_c = \{n : y_n = c\}$ be the set of dataset indices belonging to class c. A class-balanced row is constructed by sampling one example from each class. For imbalanced datasets, minority classes may be sampled with replacement, while majority classes may be subsampled across rows, so that every class contributes equally to each row.

Each row is shuffled and split into groups of size K. When $K \le C$, complete groups formed from a class-balanced row contain distinct labels by construction, because each class appears at most once in the row. This encourages diverse multi-class mixtures.

A.2 Incomplete groups and row-size preservation

Suppose a row contains N_{row} samples. Write $N_{\mathrm{row}} = K \left\lfloor \frac{N_{\mathrm{row}}}{K} \right\rfloor + r$, where $0 \le r < K$. TreemapMix processes the first $K \left\lfloor \frac{N_{\mathrm{row}}}{K} \right\rfloor$ samples as complete groups of size K. If $r > 0$, we select $K - r$ distinct image indices uniformly from the earlier, already processed part of the same row and load those images again to pad the tail group. We sample images, not new classes; because a balanced row contains one image per class, these padding images have labels distinct from the tail and from one another. Rows shorter than K are passed through unmixed.

TreemapMix generates K cyclic views from the temporary group but retains only the first r outputs. Therefore, the total number of final TreemapMix samples contributed by the row is $K \left\lfloor \frac{N_{\mathrm{row}}}{K} \right\rfloor + r = N_{\mathrm{row}}$.

Thus, output count matches the constructed row stream. Retaining only $r < K$ views does not provide the full-orbit balancing guarantee.

A.3 Integer box conversion and layout fallback

The squarified treemap is first computed in normalized coordinates. Let $\bar{B}_i = (\bar{x}_i, \bar{y}_i, \bar{w}_i, \bar{h}_i)$ denote the normalized rectangle for slot i. We convert it to an integer pixel box $B_i = (x_i^{(0)}, y_i^{(0)}, x_i^{(1)}, y_i^{(1)})$. Because independent rounding can introduce gaps or overlaps, shared boundaries are snapped consistently. Including left/top boundaries and excluding right/bottom boundaries, a validated conversion satisfies $B_i \cap B_j = \emptyset$ for $i \ne j$ and $\bigcup_{i=1}^{K} B_i = [0, W) \times [0, H)$.

A converted layout is considered valid if every box has positive area. Equivalently, for integer boxes, every box must have width and height at least one pixel. If the treemap layout fails this validity check, TreemapMix falls back to a stripe layout. The stripe layout partitions the canvas

along one axis according to the sorted weights $\lambda_1, \ldots, \lambda_K$, then applies the same integer-boundary snapping procedure. For vertical stripes, $K \leq W$ and $H \geq 1$ permit a positive-area full-canvas tiling. Boundaries are adjusted to give each stripe at least one pixel. Consequently, actual area fractions only approximate the sampled weights; minimum-width adjustments can exceed nearest-integer rounding error. Target weights remain the sampled weights.

A.4 Symmetry transformations

For square images, TreemapMix samples from the dihedral group

$$D_4 = \{\text{identity}, 90^\circ \text{ rotation}, 180^\circ \text{ rotation}, 270^\circ \text{ rotation}, \\ \text{horizontal flip}, \text{vertical flip}, \text{transpose}, \text{anti-transpose}\}.$$

For non-square images, TreemapMix uses the shape-preserving subset

$$\{\text{identity}, 180^\circ \text{ rotation}, \text{horizontal flip}, \text{vertical flip}\}.$$

These transformations preserve region areas, so they increase spatial diversity without changing the TreemapMix target.

A.5 Cyclic views and slot exposure

Cyclic views are a tractable alternative to enumerating all $K!$ image-to-slot assignments. If an epoch contains G complete TreemapMix groups, full permutation enumeration would produce $K!G$ mixed images, increasing the sample count by $\frac{K!G}{KG} = (K-1)!$. In contrast, cyclic TreemapMix produces exactly K views per group, giving KG mixed images and preserving the number of training samples after class-balanced row construction.

Cyclic permutation is designed to decouple source labels from fixed spatial slots. Although any individual mixed view may assign a class to a small or large region, the full cyclic orbit exposes every source image to every slot exactly once. Therefore, when the source labels in a group are distinct, every participating class is paired once with every area weight and, for PL training, every rank position induced by the sorted slot weights. Appendix Lemma 3 formalizes the resulting slot-exposure balance.

B Soft TreemapMix Cross-Entropy Derivations

We use the sparse target notation from Section 3.2. For one TreemapMix sample, let $\mathcal{T} = \{(c_i, \eta_i)\}_{i=1}^{K}$, where $\eta_i \geq 0$ and $\sum_{i=1}^{K} \eta_i = 1$, where c_i is the class assigned to slot i and η_i is the corresponding slot weight. For cyclic view s, we have $c_i = y_{\pi_s(i)}$ and $\eta_i = \lambda_i$.

B.1 KL derivation

The sparse target distribution is $q_c = \sum_{i=1}^{K} \eta_i \mathbf{1}[c_i = c]$. It is a valid probability distribution because

$$\sum_{c=1}^{C} q_c = \sum_{c=1}^{C} \sum_{i=1}^{K} \eta_i \mathbf{1}[c_i = c] = \sum_{i=1}^{K} \eta_i \sum_{c=1}^{C} \mathbf{1}[c_i = c] = \sum_{i=1}^{K} \eta_i = 1.$$

Hence $q = (q_1, \ldots, q_C)$ lies on the probability simplex.

The KL divergence from q to p_θ is

$$D_{\mathrm{KL}}(q \| p_\theta) = \sum_{c=1}^{C} q_c \log \frac{q_c}{p_\theta(c \mid \tilde{x})} = \sum_{c=1}^{C} q_c \log q_c - \sum_{c=1}^{C} q_c \log p_\theta(c \mid \tilde{x}).$$

The first term is independent of θ. Thus, minimizing the KL divergence is equivalent to minimizing soft cross-entropy. Substituting the sparse target gives

$$\begin{aligned} \mathcal{L}_{\text{TM-CE}} &= -\sum_{c=1}^{C} \left(\sum_{i=1}^{K} \eta_i \mathbf{1}[c_i = c] \right) \log p_\theta(c \mid \tilde{x}) \\ &= -\sum_{i=1}^{K} \eta_i \log p_\theta(c_i \mid \tilde{x}), \end{aligned}$$

which is the sparse form in (5). For a mini-batch of size B,

$$\mathcal{L}_{\text{TM-CE}} = -\frac{1}{B} \sum_{b=1}^{B} \sum_{i=1}^{K} \eta_{b,i} \log p_\theta(c_{b,i} \mid \tilde{x}_b).$$

B.2 Gradient derivation

For one TreemapMix sample, write

$$\mathcal{L}_{\text{TM-CE}} = -\sum_{c=1}^{C} q_c \log p_c, \qquad p_c = \frac{\exp(z_c)}{\sum_{j=1}^{C} \exp(z_j)}.$$

Since $\sum_{c=1}^{C} q_c = 1$, the log-softmax identity gives

$$\begin{aligned} \log p_c &= z_c - \log \sum_{j=1}^{C} \exp(z_j), \\ \mathcal{L}_{\text{TM-CE}} &= -\sum_{c=1}^{C} q_c z_c + \log \sum_{j=1}^{C} \exp(z_j), \\ \frac{\partial \mathcal{L}_{\text{TM-CE}}}{\partial z_i} &= -q_i + \frac{\exp(z_i)}{\sum_{j=1}^{C} \exp(z_j)} = p_i - q_i. \end{aligned}$$

Substituting $q_i = \sum_j \eta_j \mathbf{1}[c_j = i]$ gives the sparse gradient. For a mini-batch of size B,

$$\frac{\partial \mathcal{L}_{\text{TM-CE}}}{\partial z_{b,c}} = \frac{1}{B} \left[p_\theta(c \mid \tilde{x}_b) - \sum_{i=1}^{K} \eta_{b,i} \mathbf{1}[c_{b,i} = c] \right].$$

B.3 Random cyclic-view target variance

Lemma 1 (Random cyclic-view and cyclic-average target variance). Assume distinct source labels, an unfiltered draw $w \sim \mathrm{Dirichlet}(\alpha, \ldots, \alpha)$ with $\alpha > 0$, and let $q_c^{(s)} = \sum_{i=1}^{K} \lambda_i \mathbf{1}[y_{\pi_s(i)} = c]$ be the TreemapMix-SCE target mass for class c in cyclic view s. Conditioned on the source labels, if c is a participating class and s is sampled uniformly from $\{0, \ldots, K-1\}$ independently of w, then

$$\mathbb{E}_{w,s}[q_c^{(s)}] = \frac{1}{K}, \qquad \mathrm{Var}_{w,s}(q_c^{(s)}) = \frac{K-1}{K^2(K\alpha+1)}.$$

Moreover, averaging over the full cyclic orbit gives $\frac{1}{K}\sum_{s=0}^{K-1} q_c^{(s)} = \frac{1}{K}$, so the cyclic-average target mass has zero variance conditioned on the source labels. For an absent class, $q_c^{(s)} = 0$ for all s.

Proof. Random cyclic view. Assume the K source labels are distinct, and let $c = y_k$ be a participating class. For each cyclic view s, class c appears in exactly one slot. Since the cyclic map visits every slot exactly once as s ranges from 0 to $K-1$, sampling s uniformly makes the slot index containing class c uniformly distributed over $\{1, \ldots, K\}$. Therefore, $q_c^{(s)} = \lambda_I$, where $I \sim \mathrm{Uniform}\{1, \ldots, K\}$. Because $(\lambda_1, \ldots, \lambda_K)$ is a sorted permutation of $(w_1, \ldots, w_K)$, a uniformly selected sorted component λ_I has the same marginal distribution as a uniformly selected unsorted component w_J. By exchangeability of the symmetric Dirichlet distribution, this marginal distribution is the same as that of any component w_j. Hence $\mathbb{E}_{w,s}[q_c^{(s)}] = \mathbb{E}[w_j] = \frac{1}{K}$. For a symmetric Dirichlet distribution with concentration parameter α in each coordinate, $\mathrm{Var}(w_j) = \frac{K-1}{K^2(K\alpha+1)}$. Therefore, $\mathrm{Var}_{w,s}(q_c^{(s)}) = \mathrm{Var}(w_j) = \frac{K-1}{K^2(K\alpha+1)}$.

Full cyclic orbit. The cyclic-average identity follows from Lemma 3. When labels are distinct, $m_c = 1$ for every participating class, so $\frac{1}{K}\sum_{s=0}^{K-1} q_c^{(s)} = \frac{1}{K}$. Since this equality holds for every Dirichlet sample, the conditional variance of the cyclic-average target mass is zero. If c is absent from the group, then no slot carries class c, and $q_c^{(s)} = 0$ for all s.

Fixed-logit interpretation. If logits are fixed, then $\frac{\partial \mathcal{L}_{\text{TM-CE}}}{\partial z_c} = p_c - q_c^{(s)}$ has the same variance as $q_c^{(s)}$, since p_c is held fixed while w and s vary. This is a fixed-logit target-side calculation, not a formula for the stochastic network-gradient variance when logits depend on the mixed input. Layout rejection and uniform fallback alter the weight distribution, so the exact Dirichlet variance need not hold for the filtered training sampler. The full-orbit identity holds for any normalized weights reused across all views.

□

C TreemapMix Plackett–Luce Derivations

The derivation assumes K distinct target classes. Let $r_1 \succ \cdots \succ r_K$ denote their descending order by area weight, set $L = K$, and write ρ_{r_ℓ} for the area weight assigned to class r_ℓ. Let $\mathcal{U} = \{1, \ldots, C\} \setminus \{r_1, \ldots, r_K\}$ be the absent classes, with total predicted probability mass $p_{\mathcal{U}} = 1 - \sum_{j=1}^{K} p_{r_j}$.

C.1 From the Plackett–Luce probability to TreemapMix-PL

Let the model output logits $z = f_\theta(\tilde{x}) \in \mathbb{R}^C$ and softmax probabilities $p_c = p_\theta(c \mid \tilde{x})$. For the descending ranking $r_1 \succ \cdots \succ r_L$, the top-$L$ Plackett–Luce probability with unranked residual mass is

$$P_{\mathrm{PL}}(r_1 \succ \cdots \succ r_L \mid \tilde{x}) = \prod_{\ell=1}^{L} \frac{p_{r_\ell}}{p_{\mathcal{U}} + \sum_{j=\ell}^{L} p_{r_j}}.$$

The corresponding unweighted ListMLE-style negative log-likelihood is

$$\mathcal{L}_{\mathrm{ListMLE}} = -\sum_{\ell=1}^{L} \log \frac{p_{r_\ell}}{p_{\mathcal{U}} + \sum_{j=\ell}^{L} p_{r_j}}.$$

TreemapMix-PL weights each ranking term by its area weight ρ_{r_ℓ}:

$$\mathcal{L}_{\text{TM-PL}} = -\sum_{\ell=1}^{L} \rho_{r_\ell} \log \frac{p_{r_\ell}}{p_{\mathcal{U}} + \sum_{j=\ell}^{L} p_{r_j}}.$$

This is the descending form in (9). This weighted stage-loss surrogate is not generally a normalized likelihood over rankings. With equal weights it is a constant rescaling of the unweighted top-K ListMLE loss. Equal-weight entries retain their source-slot order, including under uniform fallback.

C.2 Ascending implementation form

For implementation, we reverse the descending order. Define $a_i = r_{L+1-i}$, $\gamma_i = \rho_{a_i}$, and $\gamma_1 \le \gamma_2 \le \cdots \le \gamma_L$. Thus, a_1 is the weakest ranked class and a_L is the strongest. Reindexing the descending objective gives

$$\mathcal{L}_{\text{TM-PL}} = -\sum_{i=1}^{L} \gamma_i \log \frac{p_{a_i}}{p_{\mathcal{U}} + \sum_{j=1}^{i} p_{a_j}}.$$

Using

$$D_i = p_{\mathcal{U}} + \sum_{j=1}^{i} p_{a_j} = 1 - \sum_{j=i+1}^{L} p_{a_j},$$

we obtain

$$\mathcal{L}_{\text{TM-PL}} = \sum_{i=1}^{L} \gamma_i \left[\log D_i - \log p_{a_i}\right],$$

which is the ascending implementation form in (11). For a mini-batch of size B, where sample b has L_b distinct ranked classes, the corresponding batch objective is

$$\mathcal{L}_{\text{TM-PL}} = \frac{1}{B} \sum_{b=1}^{B} \sum_{i=1}^{L_b} \gamma_{b,i} \left[\log D_{b,i} - \log p_\theta(a_{b,i} \mid \tilde{x}_b)\right].$$

C.3 Gradient derivation

We differentiate the ascending objective above, holding its target weights and rank assignments fixed. Define the stronger-class probability mass $R_i = \sum_{j=i+1}^{L} p_{a_j}$ and $D_i = 1 - R_i$, and the stronger-class set $T_i = \{a_{i+1}, \ldots, a_L\}$. First, $\frac{\partial \log p_{a_i}}{\partial z_c} = \mathbf{1}[c = a_i] - p_c$. Next,

$$\begin{aligned}\frac{\partial R_i}{\partial z_c} &= \sum_{j=i+1}^{L} p_{a_j} \left(\mathbf{1}[c = a_j] - p_c\right) \\ &= p_c \mathbf{1}[c \in T_i] - p_c R_i \\ &= p_c \left(\mathbf{1}[c \in T_i] - R_i\right).\end{aligned}$$

The denominator derivatives are therefore

$$\begin{aligned}\frac{\partial D_i}{\partial z_c} &= -\frac{\partial R_i}{\partial z_c} = p_c \left(R_i - \mathbf{1}[c \in T_i]\right), \\ \frac{\partial \log D_i}{\partial z_c} &= \frac{p_c \left(R_i - \mathbf{1}[c \in T_i]\right)}{D_i}.\end{aligned}$$

For the i-th term $\ell_i = \gamma_i[\log D_i - \log p_{a_i}]$, we have

$$\begin{aligned}\frac{\partial \ell_i}{\partial z_c} &= \gamma_i \left[\frac{p_c \left(R_i - \mathbf{1}[c \in T_i]\right)}{D_i} - \mathbf{1}[c = a_i] + p_c\right] \\ &= \gamma_i \left[\frac{p_c \left(R_i - \mathbf{1}[c \in T_i] + D_i\right)}{D_i} - \mathbf{1}[c = a_i]\right] \\ &= \gamma_i \left[\frac{p_c \left(1 - \mathbf{1}[c \in T_i]\right)}{D_i} - \mathbf{1}[c = a_i]\right],\end{aligned}$$

because $R_i + D_i = 1$. Summing over i gives

$$\frac{\partial \mathcal{L}_{\text{TM-PL}}}{\partial z_c} = \sum_{i=1}^{L} \gamma_i \left[\frac{p_c \left(1 - \mathbf{1}[c \in T_i]\right)}{D_i} - \mathbf{1}[c = a_i]\right],$$

which is the gradient in (13).

C.4 Ordinal gradient structure

The gradient above has a simple ordinal interpretation.

Unranked class

Let $u \notin \{a_1, \ldots, a_L\}$. Then $\mathbf{1}[u \in T_i] = 0$ and $\mathbf{1}[u = a_i] = 0$ for all i, so $\frac{\partial \mathcal{L}_{\text{TM-PL}}}{\partial z_u} = p_u \sum_{i=1}^{L} \frac{\gamma_i}{D_i} \geq 0$. Thus, direct gradient descent on logits suppresses absent classes. Updates through shared network parameters need not move each individual logit in that direction.

Ranked class

Let $c = a_m$ for some $m \in \{1, \ldots, L\}$. For $i < m$, class a_m belongs to the stronger set T_i, so those terms vanish. For $i = m$, the contribution is $\gamma_m \left(\frac{p_{a_m}}{D_m} - 1\right)$. For $i > m$, class a_m is not in T_i and is not equal to a_i, so the remaining contribution is $p_{a_m} \sum_{i=m+1}^{L} \frac{\gamma_i}{D_i}$. Therefore,

$$\frac{\partial \mathcal{L}_{\text{TM-PL}}}{\partial z_{a_m}} = \gamma_m \left(\frac{p_{a_m}}{D_m} - 1\right) + p_{a_m} \sum_{i=m+1}^{L} \frac{\gamma_i}{D_i}.$$

For the strongest class a_L, the sum is empty and $D_L = 1$, so $\frac{\partial \mathcal{L}_{\text{TM-PL}}}{\partial z_{a_L}} = \gamma_L (p_{a_L} - 1) \leq 0$. Thus, direct logit gradient descent promotes the strongest ranked class unless $p_{a_L} = 1$, suppresses unranked classes, and regulates intermediate ranked classes through competition with stronger classes.

C.5 Proof of the Dirichlet order-statistics result

Lemma 2 (Dirichlet order statistics and ranking emphasis). Assume distinct target labels and unfiltered symmetric Dirichlet weights with $\alpha > 0$, so that $L = K$ and $\gamma_i = \lambda_i$. Let $\mu_i(\alpha) = \mathbb{E}[\lambda_i]$ denote the expected i-th sorted Dirichlet weight. Holding logits and rank assignments fixed while varying the weights, the expected TreemapMix-PL gradients satisfy

$$\mathbb{E}_w\left[\frac{\partial \mathcal{L}_{\text{TM-PL}}}{\partial z_{a_m}}\right] = \mu_m(\alpha)\left(\frac{p_{a_m}}{D_m} - 1\right) + p_{a_m}\sum_{i=m+1}^{K}\frac{\mu_i(\alpha)}{D_i},$$

$$\mathbb{E}_w\left[\frac{\partial \mathcal{L}_{\text{TM-PL}}}{\partial z_u}\right] = p_u\sum_{i=1}^{K}\frac{\mu_i(\alpha)}{D_i}, \qquad u \notin \{a_1, \ldots, a_K\}.$$

Proof. Expected gradient. Assume the K target labels are distinct, so that $L = K$ and the ascending PL weights satisfy $\gamma_i = \lambda_i$. Let $\mu_i(\alpha) = \mathbb{E}[\lambda_i]$ denote the expected i-th sorted Dirichlet weight. Holding logits and rank assignments fixed while varying the weights, D_i and p_c are fixed, and the ranked-class gradient is linear in λ_i. Taking expectations term by term replaces each λ_i by $\mu_i(\alpha)$, giving both identities in the lemma.

The expected fixed-logit gradient therefore depends on α through $\mu_i(\alpha)$. This linearity argument does not establish monotonicity of individual gaps between order statistics.

Large-concentration limit. For $\alpha \to \infty$, each coordinate has mean $1/K$ and variance $(K - 1)/(K^2(K\alpha + 1)) \to 0$. A union bound implies $\max_j |w_j - 1/K| \to 0$ in probability, and sorting preserves this convergence. Thus the loss tends to $1/K$ times the unweighted top-K objective for fixed logits and rank assignments.

Scope. For filtered sampling the same linearity uses moments of the actual accepted/fallback distribution, rather than unfiltered Dirichlet moments. These calculations hold predictions fixed; they do not describe the full training-gradient expectation when predictions vary with the mixed input.

□

D Proof of Cyclic Permutation Balancing

Lemma 3 (Cyclic permutation balances slot and target exposure) Let the normalized slot weights satisfy $\sum_{i=1}^{K} \lambda_i = 1$, and define

$$q_c^{(s)} = \sum_{i=1}^{K} \lambda_i \mathbf{1}[y_{\pi_s(i)} = c], \qquad m_c = \sum_{k=1}^{K} \mathbf{1}[y_k = c]. \tag{16}$$

Then, for every class c and slot i,

$$\frac{1}{K} \sum_{s=0}^{K-1} \mathbf{1}[y_{\pi_s(i)} = c] = \frac{m_c}{K}, \tag{17}$$

and consequently

$$\frac{1}{K} \sum_{s=0}^{K-1} q_c^{(s)} = \frac{m_c}{K}. \tag{18}$$

Proof. Slot exposure. For cyclic view s, slot i receives source image $x_{\pi_s(i)}$ and label $y_{\pi_s(i)}$, where $\pi_s(i) = 1 + \big((i + s - 1) \bmod K\big)$. For any fixed slot i, as s ranges from 0 to $K - 1$, the index $\pi_s(i)$ visits every source position exactly once. Therefore, for any class c, $\sum_{s=0}^{K-1} \mathbf{1}[y_{\pi_s(i)} = c] = \sum_{k=1}^{K} \mathbf{1}[y_k = c] = m_c$. Dividing by K gives the slot-balancing identity $\frac{1}{K} \sum_{s=0}^{K-1} \mathbf{1}[y_{\pi_s(i)} = c] = \frac{m_c}{K}$, which proves (17).

Target exposure. The soft target mass assigned to class c in cyclic view s is $q_c^{(s)} = \sum_{i=1}^{K} \lambda_i \mathbf{1}[y_{\pi_s(i)} = c]$. Averaging over cyclic views gives

$$\begin{aligned}
\frac{1}{K} \sum_{s=0}^{K-1} q_c^{(s)} &= \frac{1}{K} \sum_{s=0}^{K-1} \sum_{i=1}^{K} \lambda_i \mathbf{1}[y_{\pi_s(i)} = c] \\
&= \frac{1}{K} \sum_{i=1}^{K} \lambda_i \sum_{s=0}^{K-1} \mathbf{1}[y_{\pi_s(i)} = c] \\
&= \frac{1}{K} \sum_{i=1}^{K} \lambda_i m_c \\
&= \frac{m_c}{K},
\end{aligned}$$

because $\sum_{i=1}^{K} \lambda_i = 1$. This proves (18).

When the K source labels are distinct, $m_c = 1$ for every participating class and $m_c = 0$ for every absent class. Therefore, each participating class appears exactly once in every slot across the K cyclic views. Since slots are associated with sorted weights and rank positions, cyclic permutation also prevents any participating class from being permanently tied to a fixed TreemapMix weight or rank. The PL denominator terms remain prediction-dependent, so this balancing does not remove ordinal competition; it only removes persistent source-label-to-slot bias.

□

E Implementation Details

E.1 Experimental Protocol and Hyperparameters

Tables 8, 9, and 10 summarize the ImageNet-1K experimental protocol and baseline hyperparameters. Table 8 summarizes the schedule-level adaptations applied to the reference `timm` recipes for the main 110-epoch comparison. Tables 9 and 10 separate data-augmentation settings from optimizer and training settings for readability.

The baseline recipes are adapted from established `timm` ImageNet-1K training configurations [42, 43]. Because our main study uses a shorter 110-epoch budget than many reference recipes, we adapt the training recipe: we disable `aug_repeats`, adjust the learning rate, disable gradient clipping, and keep the remaining regularization settings fixed across methods. All methods therefore share the same architecture, data budget, optimizer schedule, and evaluation protocol. The comparison isolates augmentation and loss effects, but does not claim per-baseline optimality.

Runs were executed on H20 nodes with eight GPUs and 2 TB system RAM, with independent two-GPU jobs colocated by a custom scheduler. Across reported main and appendix experiments, training used approximately 5.5k GPU-hours, while peak observed host-memory use remained below 400 GB per node.

The main ImageNet-1K comparison uses a global batch size of 1024. We omit epochs and warmup epochs from Tables 9 and 10 because those values are held fixed across runs. The augmentation table lists the ImageNet-1K augmentation recipes used in the main comparisons.

Table 8: Adaptations to the reference `timm` ImageNet-1K recipes used for the main 110-epoch comparison. We shorten the original long schedules, disable augmentation repeats, switch to `bfloat16`, and retune the learning rate; all other recipe settings are kept unchanged unless listed.

Model	Setting	Original	New
ViT-Betwixt			
	`dtype`	`float16`	`bfloat16`
	`aug_repeats`	3.0	0.0
	`warmup_epochs`	20	10
	`epochs`	600	110
	`lr`	1.2e-3	1.5e-3
ViT-Medium			
	`dtype`	`float16`	`bfloat16`
	`aug_repeats`	3.0	0.0
	`clip_grad`	5.0	`null`
	`warmup_epochs`	20	10
	`epochs`	600	110
	`lr`	1.2e-3	1.5e-3
ViT-Little			
	`dtype`	`float16`	`bfloat16`
	`aug_repeats`	3.0	0.0
	`clip_grad`	5.0	`null`
	`warmup_epochs`	20	10
	`epochs`	600	110
	`lr`	1.2e-3	1.5e-3
ViT-Wee			
	`dtype`	`float16`	`bfloat16`
	`aug_repeats`	3.0	0.0
	`clip_grad`	5.0	`null`
	`warmup_epochs`	20	10
	`epochs`	450	110
	`lr`	1.2e-3	1.5e-3

Table 9: ImageNet-1K augmentation settings. All recipes except No Aug include single-image transforms (cropping, flipping, color/AutoAugment-style transforms, and random erasing). p is the application probability; α denotes the method's mixing parameter. Settings were checked against archived run configurations and the historical training implementation. OpenMixup options beyond α and p are implementation defaults without recorded overrides. The archived No Aug label disables single-image transforms but retains Mixup+CutMix with equal selection probabilities. RICAP records the configured close parameter; its effective cutoff depends on the configured training horizon. All TreemapMix variants use aspect-ratio filtering with maximum ratio 20.

Method	Mixing settings
No Aug	Single-image transforms off; Mixup+CutMix with $\alpha = 0.8/1.0$, $p = 1$
Single-Image Aug	No mixing
Mixup	$\alpha = 0.8$, $p = 1$
CutMix	$\alpha = 1.0$, $p = 1$
Mixup+CutMix	$\alpha = 0.8/1.0$, $p = 1$; choose either method with probability 0.5
RICAP	$p = 0.5$; `close_epochs`=10; center $[0.5, 1.5]$; scale $[0.8, 1.2]$
FMix	$\alpha = 0.4$, $p = 1$; spectral decay 3, hard mask
GridMix	$\alpha = 0.1$, $p = 1$; 20 holes; hole/cut aspect and cut-area ratios 1
ResizeMix	$\alpha = 0.1$ (unused); $p = 1$; scale uniform in $[0.1, 0.8]$, nearest interpolation
SaliencyMix	$\alpha = 0.25$, $p = 1$
SmoothMix	$\alpha = 0.6$, $p = 1$
TokenMix	$\alpha = 0.4$, $p = 1$; block mask, minimum 14 tokens
TLA	$\alpha = 0.2$, $p = 1$; patch size 16
TreemapMix-SCE	$K = 4$; cosine $\alpha : 0.1 \rightarrow 0.5$
TreemapMix-PL	$K = 6$; cosine $\alpha : 0.1 \rightarrow 0.5$
TreemapMix-mixed	$K = 4$; cosine $\alpha : 0.1 \rightarrow 0.5$; $\beta = 0.5$

Table 10: ViT baseline optimizer and training hyperparameters for the main comparison. Opt. denotes optimizer, LR learning rate, WD weight decay, and EMA exponential moving average of model weights. NAdamW is Nesterov-accelerated Adam with decoupled weight decay. DropPath is the stochastic-depth probability.

Model	Opt.	Base LR	LR Decay	WD	DropPath	EMA
ViT-Wee	NAdamW	1.5e-3	cosine	0.06	0.2	yes
ViT-Little	NAdamW	1.5e-3	cosine	0.06	0.2	no
ViT-Medium	NAdamW	1.5e-3	cosine	0.06	0.2	no
ViT-Betwixt	NAdamW	1.5e-3	cosine	0.08	0.2	no

E.2 Additional COCO transfer results

This subsection supplements the main transfer analysis with the fine-tuning hyperparameters in Table 11 and box/mask AP50 results in Table 12. AP50 evaluates average precision at an intersection-over-union threshold of 0.50, complementing the main table's AP averaged over thresholds from 0.50 to 0.95.

Table 11: Hyperparameters for the object-detection transfer experiments with a ViTDet-style adapter and Mask R-CNN on COCO. Embed Dim is the embedding dimension, Heads is the attention-head count, MLP Ratio is the multilayer perceptron expansion ratio, and LR is the learning rate.

Model	Embed Dim	Depth	Heads	MLP Ratio	Drop Path	LR	Layer Decay
ViT-Wee	256	14	4	5.0	0.1	6.0e-4	0.7
ViT-Betwixt	640	12	10	4.0	0.1	8.0e-4	0.7

At this fixed overlap threshold, PL improves box/mask AP50 over Mixup+CutMix by 1.23/0.95 points on ViT-Wee and 1.12/0.89 points on ViT-Betwixt (Table 12). Thus, the transfer gains also appear under the less stringent AP50 criterion, complementing the main multi-threshold AP results.

Table 12: COCO transfer AP50 after ViTDet-style Mask R-CNN fine-tuning from ImageNet-1K checkpoints. In this transfer setting, TreemapMix-PL obtains the best box and mask AP50 for both ViT-Wee and ViT-Betwixt, while TreemapMix-SCE also improves over Mixup+CutMix and RICAP. Values are mean ± standard deviation over three seeds; higher is better, and bold marks the best mean in each column.

	ViT-Wee		ViT-Betwixt	
Method	Box AP50 ↑	Mask AP50 ↑	Box AP50 ↑	Mask AP50 ↑
Mixup+CutMix	39.67 ± 0.01	35.64 ± 0.09	46.46 ± 0.19	42.59 ± 0.30
RICAP	38.68 ± 0.24	34.82 ± 0.20	44.91 ± 0.04	41.05 ± 0.20
TreemapMix-SCE	40.37 ± 0.19	36.18 ± 0.19	47.06 ± 0.15	43.12 ± 0.13
TreemapMix-PL	**40.90 ± 0.01**	**36.59 ± 0.13**	**47.58 ± 0.27**	**43.48 ± 0.14**

E.3 Loss functions in code

We include concise PyTorch reference implementations of TreemapMix-SCE (Figure 4), TreemapMix-PL (Figure 5), and the mixed-loss variant (Figure 6) for the distinct-label mathematical objectives. The PL listing is a numerically stable reference, not a reconstruction of every branch of the experimental training code.

```python
import torch.nn.functional as F

def treemapmix_sce_loss(logits, labels, weights):
    """
    logits: model logits, shape (B, C)
    labels: target class indices, shape (B, K)
    weights: target area weights, shape (B, K)
    Each pair (labels[b, i], weights[b, i]) represents one
    TreemapMix region for sample b.
    """
    log_probs = F.log_softmax(logits, dim=1)
    target_log_probs = log_probs.gather(dim=1, index=labels)
    losses = -(weights * target_log_probs).sum(dim=1)
    return losses.mean()
```

Figure 4: TreemapMix-SCE. The loss matches the area-weighted soft target induced by the TreemapMix regions.

```python
import torch

def treemapmix_pl_loss(logits, labels, weights):
    # logits: (B, C); labels and weights: (B, K)
    # Distinct labels; ascending area order; weights sum to 1.
    ranked = logits.gather(1, labels)
    absent = torch.ones_like(logits, dtype=torch.bool)
    absent.scatter_(1, labels, False)
    absent_lse = torch.logsumexp(
        logits.masked_fill(~absent, -torch.inf), dim=1)
    prefix_lse = torch.logcumsumexp(ranked, dim=1)
    denom_lse = torch.logaddexp(
        absent_lse[:, None], prefix_lse)
    return (weights * (denom_lse - ranked)).sum(1).mean()
```

Figure 5: Numerically stable reference for the distinct-label TreemapMix-PL objective. Log-domain remaining-set normalization avoids subtracting probabilities from one.

```python
def treemapmix_mixed_loss(logits, labels, weights, beta=0.5):
    sce = treemapmix_sce_loss(logits, labels, weights)
    pl = treemapmix_pl_loss(logits, labels, weights)
    return beta * sce + (1.0 - beta) * pl
```

Figure 6: TreemapMix-mixed, using the functions in Figures 4 and 5. The coefficient β matches the notation in the main text.

E.4 Runtime overhead

For the configurations in this benchmark, augmentations are applied in the data loader. Table 5 isolates data-loader throughput and host-memory usage for six augmentation configurations. The benchmark measures CPU-side loading and augmentation without model forward or backward passes. Peak resident set size (RSS) and unique set size (USS) are summed over the loader process tree. RSS can count shared pages more than once, while USS measures private memory. Memory is reported in GiB.

Relative to the single-image recipe (1282.8 images/s), TreemapMix-SCE at $K = 4$ reaches 1039.4 images/s, a 19.0% throughput reduction; TreemapMix-PL at $K = 6$ reaches 954.3 images/s, a 25.6% reduction. The corresponding peak USS increases from 17.76 GiB to 20.16 and 21.74 GiB. Mixup+CutMix reaches 1223.4 images/s, a 4.6% reduction. The RICAP configuration reaches 364.8 images/s with 25.18 GiB peak USS. TreemapMix therefore adds measurable host-side cost.

The training configuration uses a prefetch factor of 2, which allows CPU preparation to overlap GPU computation. The isolated loader benchmark does not measure how much of this overhead is hidden during training, so it does not establish an end-to-end per-epoch timing comparison.

F Additional Experiments

F.1 Full ImageNet-1K Results

Tables 1 and 2 report the full ImageNet-1K main-comparison Top-1 accuracy and ECE results in the main text, respectively. Figure 2 summarizes the same results graphically.

All 16 configurations are also evaluated using negative log-likelihood (NLL) and Brier score in Tables 13 and 14. SCE has the lowest mean NLL on all four backbones; Brier differences are small and sometimes favor the mixed objective. These complementary metrics preserve the broader confidence-quality evaluation beyond ECE.

Table 13: ImageNet-1K negative log-likelihood for all 16 configurations under the matched 110-epoch protocol. Values are mean ± one sample standard deviation over three seeds. Bold marks the lowest mean in each backbone.

Method	ViT-Wee ↓	ViT-Little ↓	ViT-Medium ↓	ViT-Betwixt ↓
Single-Image Aug	0.923 ± 0.006	0.869 ± 0.008	0.864 ± 0.001	0.855 ± 0.001
No Aug	1.120 ± 0.009	1.137 ± 0.005	1.154 ± 0.006	1.134 ± 0.008
Mixup+CutMix	1.015 ± 0.011	0.896 ± 0.002	0.854 ± 0.008	0.811 ± 0.004
Mixup	1.012 ± 0.004	0.904 ± 0.002	0.862 ± 0.002	0.820 ± 0.002
CutMix	0.994 ± 0.011	0.870 ± 0.004	0.840 ± 0.005	0.787 ± 0.005
RICAP	0.893 ± 0.004	0.846 ± 0.007	0.870 ± 0.008	0.857 ± 0.003
FMix	0.965 ± 0.004	0.850 ± 0.001	0.821 ± 0.002	0.793 ± 0.001
GridMix	0.994 ± 0.005	0.893 ± 0.002	0.865 ± 0.006	0.837 ± 0.006
ResizeMix	0.967 ± 0.004	0.851 ± 0.007	0.816 ± 0.007	0.784 ± 0.006
SaliencyMix	0.963 ± 0.005	0.855 ± 0.003	0.824 ± 0.007	0.791 ± 0.002
SmoothMix	1.008 ± 0.009	0.878 ± 0.004	0.840 ± 0.002	0.803 ± 0.004
TokenMix	0.993 ± 0.011	0.870 ± 0.005	0.829 ± 0.004	0.796 ± 0.004
TLA	0.956 ± 0.010	0.842 ± 0.008	0.806 ± 0.003	0.775 ± 0.001
TreemapMix-mixed	0.862 ± 0.008	0.778 ± 0.011	0.739 ± 0.005	0.728 ± 0.003
TreemapMix-PL	0.881 ± 0.012	0.803 ± 0.001	0.773 ± 0.002	0.766 ± 0.008
TreemapMix-SCE	$\mathbf{0.857 \pm 0.003}$	$\mathbf{0.775 \pm 0.002}$	$\mathbf{0.735 \pm 0.004}$	$\mathbf{0.714 \pm 0.007}$

F.2 CIFAR-100 Results

We evaluate all 16 augmentation settings on CIFAR-100 with the same four ViT backbones and 256-pixel inputs. The matched schedule comprises 210 total epochs, including 10 warmup epochs. TreemapMix uses the settings from the primary comparison: $K = 4$ for SCE and the mixed objective, $K = 6$ for PL, and a cosine α schedule from 0.1 to 0.5. Figure 7 summarizes Top-1 accuracy and ECE with 15 bins; Tables 15 and 16 give all numerical results.

TreemapMix-SCE has the lowest ECE on all four backbones, ranging from 1.62 to 1.87 percentage points. TreemapMix-PL has the highest Top-1 accuracy on ViT-Wee, Medium, and Betwixt, reaching 75.72%, 75.97%, and 76.39%, respectively. On ViT-Little, ResizeMix reaches 75.54% compared with 75.42% for PL. The mixed objective does not consistently improve on either individual objective. These results retain the accuracy–calibration trade-off, without a uniform accuracy advantage across every model.

Table 14: ImageNet-1K Brier score for all 16 configurations under the matched 110-epoch protocol. Values are mean ± one sample standard deviation over three seeds. Bold marks the lowest mean in each backbone.

Method	ViT-Wee ↓	ViT-Little ↓	ViT-Medium ↓	ViT-Betwixt ↓
Single-Image Aug	0.314 ± 0.002	0.294 ± 0.003	0.291 ± 0.001	0.285 ± 0.001
No Aug	0.355 ± 0.003	0.349 ± 0.002	0.344 ± 0.001	0.335 ± 0.001
Mixup+CutMix	0.342 ± 0.003	0.305 ± 0.001	0.292 ± 0.003	0.277 ± 0.002
Mixup	0.340 ± 0.002	0.308 ± 0.001	0.294 ± 0.001	0.280 ± 0.001
CutMix	0.338 ± 0.003	0.298 ± 0.002	0.287 ± 0.002	0.270 ± 0.001
RICAP	0.319 ± 0.002	0.303 ± 0.002	0.304 ± 0.002	0.299 ± 0.001
FMix	0.330 ± 0.002	0.293 ± 0.000	0.283 ± 0.001	0.273 ± 0.001
GridMix	0.335 ± 0.002	0.303 ± 0.001	0.294 ± 0.002	0.284 ± 0.001
ResizeMix	0.328 ± 0.001	0.293 ± 0.002	0.281 ± 0.002	0.269 ± 0.001
SaliencyMix	0.327 ± 0.001	0.293 ± 0.001	0.283 ± 0.002	0.272 ± 0.000
SmoothMix	0.340 ± 0.003	0.299 ± 0.001	0.288 ± 0.001	0.275 ± 0.001
TokenMix	0.336 ± 0.003	0.297 ± 0.000	0.285 ± 0.001	0.272 ± 0.001
TLA	0.327 ± 0.003	0.292 ± 0.002	0.280 ± 0.001	0.269 ± 0.000
TreemapMix-mixed	0.313 ± 0.002	$\mathbf{0.288 \pm 0.003}$	$\mathbf{0.275 \pm 0.001}$	0.269 ± 0.000
TreemapMix-PL	$\mathbf{0.313 \pm 0.003}$	0.288 ± 0.001	0.277 ± 0.001	0.271 ± 0.002
TreemapMix-SCE	0.314 ± 0.000	0.289 ± 0.001	0.276 ± 0.001	$\mathbf{0.268 \pm 0.002}$

Table 15: CIFAR-100 Top-1 accuracy (%) at 256 pixels, approximately 210 epochs including 10 warmup epochs. Mean ± sample standard deviation over three seeds; bold marks the best mean per backbone.

Method	ViT-Wee ↑	ViT-Little ↑	ViT-Medium ↑	ViT-Betwixt ↑
No Aug	65.05 ± 0.44	63.69 ± 0.81	60.72 ± 0.54	60.70 ± 0.35
Single-Image Aug	71.50 ± 0.09	71.97 ± 0.45	72.56 ± 0.35	72.16 ± 0.06
Mixup+CutMix	74.15 ± 0.47	74.96 ± 0.64	75.81 ± 0.24	75.83 ± 0.16
Mixup	73.13 ± 0.10	73.46 ± 0.34	74.18 ± 0.18	74.02 ± 0.18
CutMix	73.39 ± 0.51	74.05 ± 0.16	75.21 ± 0.22	74.92 ± 0.83
FMix	74.58 ± 0.37	74.84 ± 0.22	75.52 ± 0.22	75.40 ± 0.07
GridMix	70.58 ± 0.16	70.34 ± 0.27	71.34 ± 0.84	71.48 ± 0.22
ResizeMix	74.99 ± 0.22	$\mathbf{75.54 \pm 0.34}$	75.89 ± 0.05	76.18 ± 0.12
SaliencyMix	74.60 ± 0.14	74.64 ± 0.29	75.43 ± 0.40	75.60 ± 0.28
SmoothMix	74.42 ± 0.11	75.00 ± 0.36	75.38 ± 0.23	75.74 ± 0.28
TLA	75.42 ± 0.13	75.27 ± 0.13	75.93 ± 0.02	75.96 ± 0.18
TokenMix	74.75 ± 0.28	75.18 ± 0.24	75.36 ± 0.25	75.82 ± 0.44
RICAP	71.71 ± 0.35	72.07 ± 0.38	72.90 ± 0.27	72.26 ± 0.26
TreemapMix-SCE	75.29 ± 0.11	75.20 ± 0.36	75.72 ± 0.11	75.73 ± 0.09
TreemapMix-mixed	75.10 ± 0.37	75.38 ± 0.24	75.89 ± 0.15	75.70 ± 0.39
TreemapMix-PL	$\mathbf{75.72 \pm 0.29}$	75.42 ± 0.45	$\mathbf{75.97 \pm 0.29}$	$\mathbf{76.39 \pm 0.35}$

Table 16: CIFAR-100 ECE (15 bins, percentage points). Same runs as Table 15; bold marks the lowest mean per backbone.

Method	ViT-Wee ↓	ViT-Little ↓	ViT-Medium ↓	ViT-Betwixt ↓
No Aug	7.64 ± 0.20	6.19 ± 0.39	4.61 ± 0.32	5.01 ± 0.08
Single-Image Aug	3.16 ± 0.17	3.75 ± 0.24	3.44 ± 0.22	3.90 ± 0.30
Mixup+CutMix	14.43 ± 0.28	14.16 ± 0.50	14.07 ± 0.19	13.93 ± 0.22
Mixup	12.67 ± 0.18	12.24 ± 0.24	12.23 ± 0.05	12.14 ± 0.24
CutMix	11.74 ± 0.02	10.85 ± 0.37	10.35 ± 0.09	10.49 ± 0.28
FMix	5.93 ± 0.14	5.57 ± 0.18	5.19 ± 0.14	4.86 ± 0.22
GridMix	3.67 ± 0.21	3.30 ± 0.26	3.68 ± 0.33	3.69 ± 0.22
ResizeMix	7.87 ± 0.66	7.41 ± 0.76	7.50 ± 0.34	7.52 ± 0.26
SaliencyMix	6.01 ± 0.49	5.35 ± 0.52	5.64 ± 0.63	5.10 ± 0.16
SmoothMix	11.58 ± 0.10	11.50 ± 0.25	11.11 ± 0.12	11.29 ± 0.33
TLA	7.14 ± 0.10	6.93 ± 0.12	6.54 ± 0.29	6.44 ± 0.31
TokenMix	10.47 ± 0.47	10.35 ± 0.37	9.89 ± 0.17	9.64 ± 0.45
RICAP	10.17 ± 0.11	10.81 ± 0.44	9.87 ± 0.14	9.99 ± 0.42
TreemapMix-SCE	$\mathbf{1.62 \pm 0.20}$	$\mathbf{1.62 \pm 0.20}$	$\mathbf{1.87 \pm 0.17}$	$\mathbf{1.79 \pm 0.08}$
TreemapMix-mixed	3.81 ± 0.20	4.60 ± 0.35	4.55 ± 0.18	4.85 ± 0.27
TreemapMix-PL	9.70 ± 0.32	10.83 ± 0.14	10.55 ± 0.11	10.41 ± 0.40

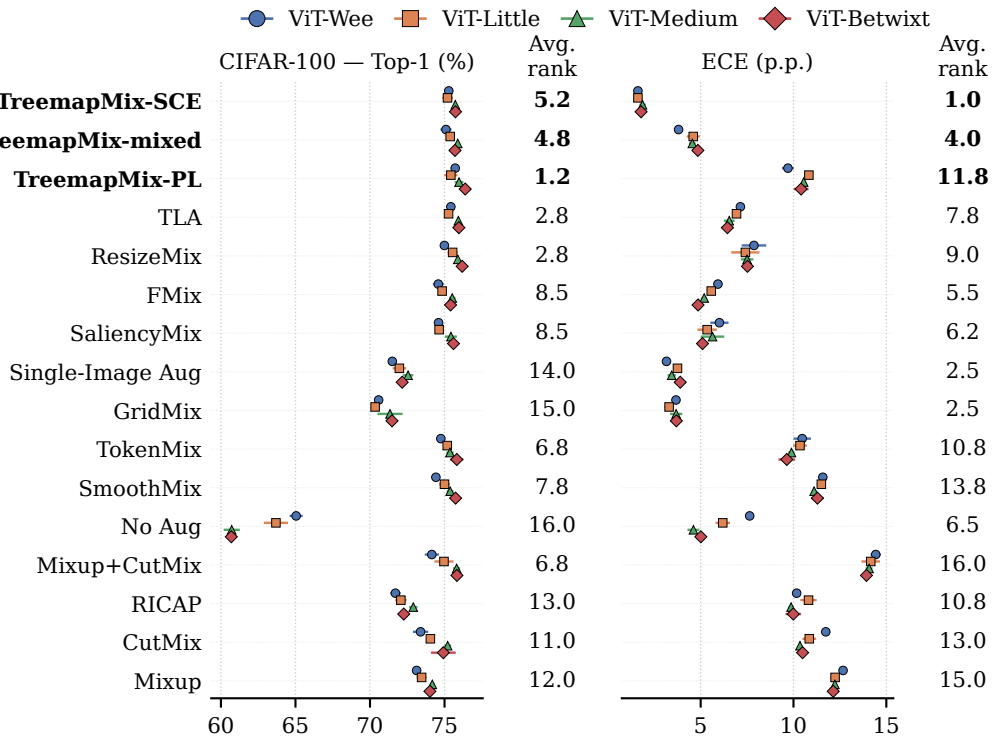


Figure 7: CIFAR-100 comparison at 256-pixel resolution under an approximately 210-epoch schedule, including 10 warmup epochs. Points show the mean over seeds 42, 43, and 44; intervals denote $\pm$ one sample standard deviation. ECE uses 15 bins and is reported in percentage points. Average ranks summarize within-backbone ranks, with rank 1 best. Complete values appear in Tables 15 and 16.

F.3 Optimizer Sensitivity

We next test whether the gains of TreemapMix persist across different optimizers on ImageNet-1K. We compare Muon [20], AdamW (Adam with decoupled weight decay) [30], and NAdamW [9, 21]. We keep the core TreemapMix hyperparameters fixed across optimizers. Because the main experiments use NAdamW with a learning rate of 1.5×10^{-3}, we retain this value for NAdamW and also use it for AdamW to keep the Adam-style optimizer comparison matched. For Muon, we use a separately tuned learning rate of 5×10^{-3}.

Table 17 compares Mixup+CutMix, TreemapMix-SCE, and TreemapMix-PL under AdamW, Muon, and NAdamW for the settings in the caption. These results should be interpreted as an optimizer-sensitivity check rather than a definitive optimizer ranking. Across the tested optimizers, the main qualitative pattern from the primary ImageNet-1K comparison is preserved: TreemapMix-PL generally gives the highest Top-1 accuracy, whereas TreemapMix-SCE gives the lowest ECE in most settings. Muon gives the strongest Top-1 accuracy in several configurations, but this gain is often accompanied by higher ECE. AdamW and NAdamW are more competitive on calibration, although neither dominates across both metrics. Overall, these single-seed results suggest that TreemapMix's accuracy and calibration trends are not specific to one optimizer, while the absolute accuracy-calibration trade-off still depends on the optimizer.

F.4 ImageNet-1K Hyperparameter Sensitivity

TreemapMix introduces two main hyperparameters: the Dirichlet concentration α and the number of mixed regions K. We study their effects with a single-seed hyperparameter sweep. The sweep identifies a stable operating regime for both TreemapMix losses: smaller α values and small-to-

Table 17: ImageNet-1K optimizer comparison under the settings in Appendix F.3, using one run with seed 42 per configuration. Accuracy is reported in percent and ECE in percentage points; bold marks the best value for each metric within each row.

		Mixup+CutMix		TreemapMix-SCE		TreemapMix-PL	
Model	Optimizer	Acc. ↑	ECE ↓	Acc. ↑	ECE ↓	Acc. ↑	ECE ↓
ViT-Wee	AdamW	75.76	8.99	77.18	**0.43**	**77.75**	5.41
	Muon	77.92	9.88	78.80	**2.04**	**79.05**	5.93
	NAdamW	76.30	9.29	77.48	**0.94**	**78.16**	5.44
ViT-Little	AdamW	78.77	8.11	79.50	**1.36**	**79.93**	5.96
	Muon	80.27	8.61	80.72	**3.05**	**80.87**	6.57
	NAdamW	78.80	7.65	79.72	**1.90**	**80.09**	6.23
ViT-Medium	AdamW	79.89	8.25	80.33	**0.87**	**80.71**	5.93
	Muon	80.98	7.86	81.42	**2.77**	**81.43**	6.49
	NAdamW	80.04	7.69	80.73	**2.09**	**81.01**	6.39
ViT-Betwixt	AdamW	81.19	8.11	81.00	**2.02**	**81.51**	7.05
	Muon	81.71	7.32	**81.85**	**3.32**	81.79	7.15
	NAdamW	81.13	7.50	81.26	**2.75**	**81.80**	7.18

medium values of K perform best, while more extreme compositions degrade both accuracy and calibration. Overall, TreemapMix is more sensitive to α than to K.

The available sweep covers $\alpha \in \{0.2, 0.4, 0.6, 0.8, 1.0, 1.5, 2.0, 3.0\}$ and $K \in \{3, \ldots, 10\}$ with the main training hyperparameters. It uses fixed K and α without aspect-ratio filtering. Figure 8 shows Top-1 accuracy and ECE across this grid, with Mixup+CutMix and single-image augmentation as references. This simplified protocol measures sensitivity; it does not reproduce the filtered, scheduled main recipe.

For Top-1 accuracy, both TreemapMix-SCE and TreemapMix-PL perform best at relatively small α values and around $K = 4$. Increasing α up to approximately 1 causes only modest degradation, while larger values lead to a clearer accuracy drop. This trend suggests that TreemapMix benefits from some imbalance among region areas rather than from nearly uniform region sizes. Larger K values also tend to reduce Top-1 accuracy, but this effect is weaker and less uniform than the effect of α. As K increases, performance varies more strongly across α values, indicating that high-composition settings are more sensitive to the choice of mixing strength.

Calibration, measured by ECE with 15 bins, follows a similar but sharper pattern. For both losses, ECE remains relatively stable up to a loss-dependent threshold and then increases substantially. For TreemapMix-SCE, this threshold occurs around $\alpha = 1$; for TreemapMix-PL, it occurs around $\alpha = 1.5$. Increasing K also tends to worsen calibration, but the effect is weaker and less systematic than the effect of α. Across the sweep, TreemapMix-PL is less sensitive to hyperparameter variation, while TreemapMix-SCE achieves the strongest calibration when α and K are chosen carefully.

These results indicate that choosing α is more important than choosing K for reliable TreemapMix performance. For calibration-focused use, the sweep supports TreemapMix-SCE with $\alpha \leq 1$ and $K \leq 5$. For accuracy-focused use, the sweep supports TreemapMix-PL with $\alpha \leq 0.4$ and $K \leq 6$. The main recipe filters layouts with extreme patch aspect ratios and schedules α from 0.1 to 0.5. These settings were not isolated in this sweep.

F.5 Qualitative Visualizations and Attribution Analysis

Figures 9, 10, and 11 visualize how TreemapMix changes with its random draw and main hyperparameters. Figure 9 holds the source images and hyperparameters fixed while varying only the random draw. Figures 10 and 11 sweep the number of regions and the Dirichlet concentration, respectively, showing how these controls change the granularity and balance of the mixed image.

We also complement the quantitative logit-alignment diagnostics with qualitative attribution maps in Figure 12. The maps are computed on the same held-out diagnostic set using the ViT-Betwixt

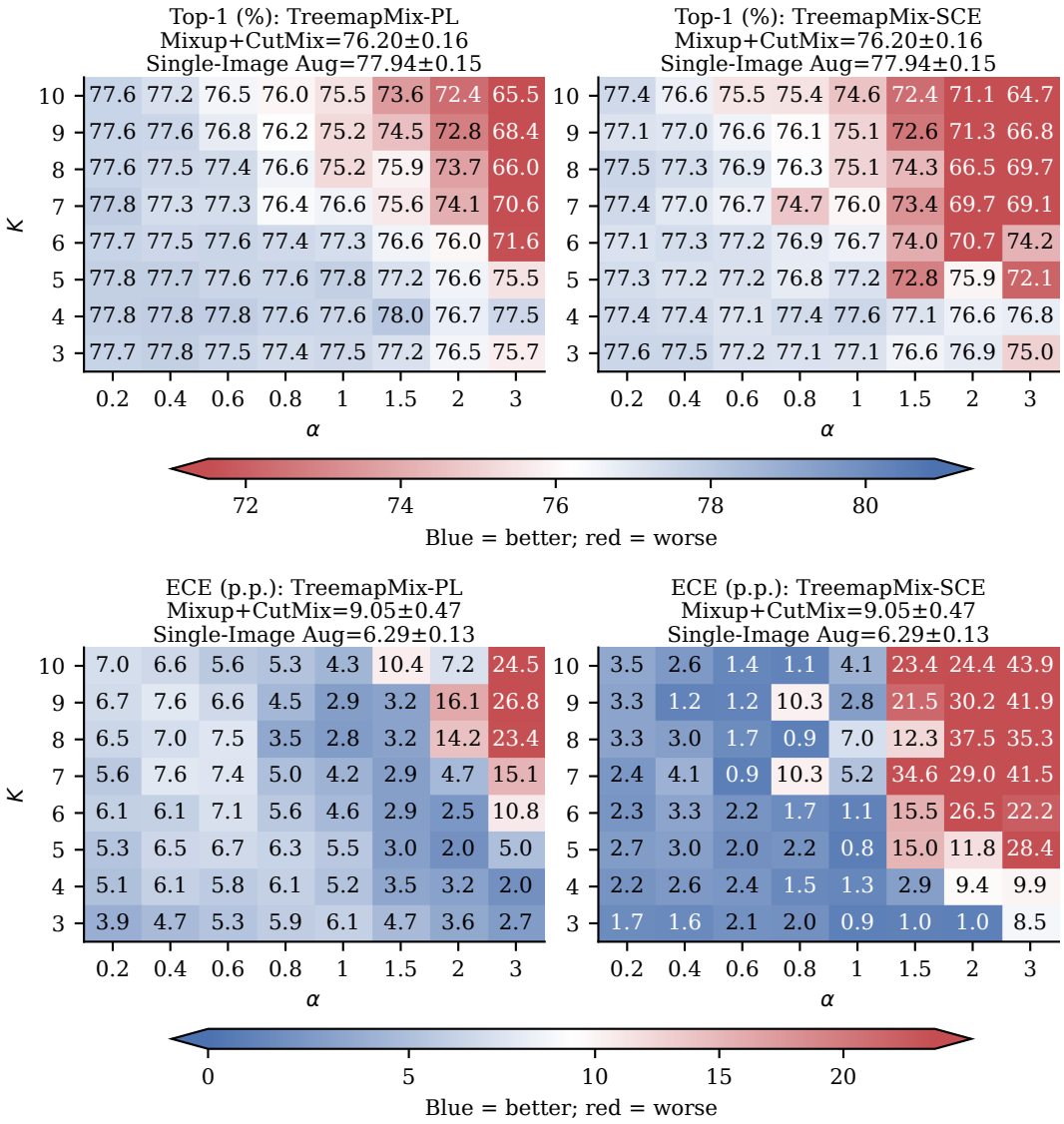


Figure 8: Appendix heatmaps for the ImageNet-1K TreemapMix hyperparameter sweep on ViT-Wee. ECE is measured using 15 bins. The top row shows Top-1 over the available (K, α) grid ($K = 3, \ldots, 10$), and the bottom row shows ECE over the same grid. The sweep uses a simplified protocol: we do not apply aspect filtering and we do not use α or K scheduling, so these absolute values are expected to be somewhat worse than the tuned main results. The heatmaps show that for best results in terms of ECE and accuracy, small K values together with moderately small α values should be used.

checkpoints. We use Gradient×Input attribution [37, 38], which multiplies the input by the gradient of the target logit with respect to the input. This provides a qualitative view of which source regions contribute most strongly to a given class logit.

Figure 12 shows different probability behavior across the losses. For boathouse, SCE assigns 0.01 while other methods round to 0.00; rounded zeros need not be exact zeros. PL assigns substantially higher probability to the largest region, consistent with ranking rather than probability matching.

TreemapMix attribution often concentrates on target regions, including the small suit and boathouse patches; baseline Cougar attributions focus more on the face. These qualitative observations complement the logit diagnostics rather than providing standalone evidence. PL shows the clearest ordinal correspondence.



Figure 9: TreemapMix randomness for fixed sources, $\alpha = 0.5$, and $K = 5$. Independent Dirichlet draws produce different compositions while preserving sources and settings.



Figure 10: Effect of source count for fixed images, $\alpha = 2.0$, and maximum aspect ratio 15.0. Increasing K from 2 to 9 gives finer-grained mixtures.

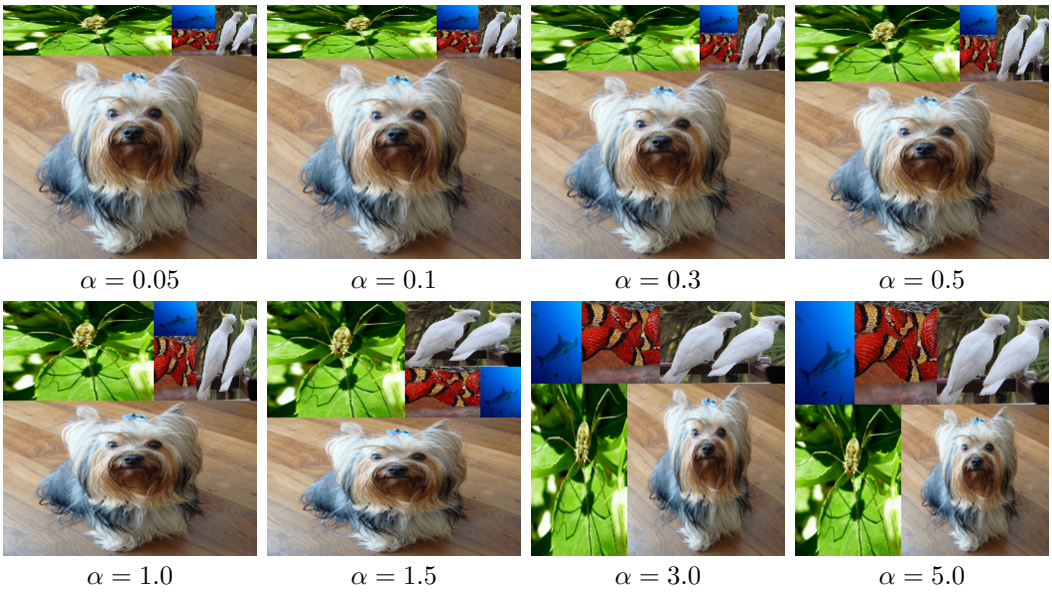


Figure 11: Effect of Dirichlet concentration for fixed images, $K = 5$, and maximum aspect ratio 15.0. Smaller α favors more unequal region areas.

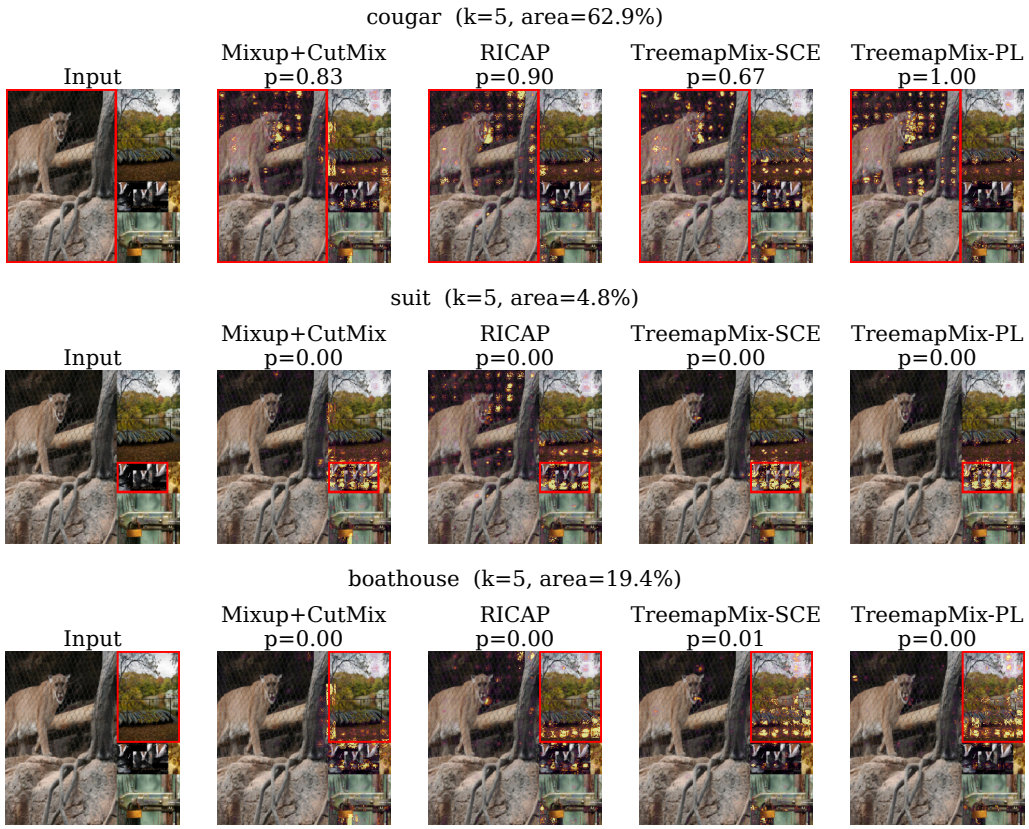


Figure 12: Gradient×Input attribution for three ImageNet-1K classes. Each row shows the boxed input and overlays for Mixup+CutMix, RICAP, SCE, and PL. Red boxes identify target patches; brighter regions indicate larger attribution. Labels give target probabilities and patch areas. TreemapMix often concentrates attribution on target regions; PL probabilities are sharper than SCE.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and introduction state the paper's main contributions and intended scope, and the later methodological, experimental, and appendix sections provide the corresponding support for these claims.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: We discuss limitations in Section 6, including area–semantic mismatch, unisolated components, computational measurement scope, long-horizon coverage, and long-tailed calibration.

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
   - While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [Yes]

Justification: Section 3 and Appendices B, C, and D give the results and proofs, with distinct-label, unfiltered-sampling, and fixed-logit assumptions stated explicitly where required.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: The paper specifies the training protocol, augmentation settings, baselines, evaluation metrics, and seed-averaged reporting used in the main experiments, with additional hyperparameter and implementation details collected in the appendix to support reproduction of the reported results.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).

(d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [Yes]

   Justification: TreemapMix augmentation, stable supervision losses, and training instructions are available at `https://github.com/Ejafa/treemapmix`. Historical experiment code remains at `https://github.com/Ejafa/LabelMix`; Appendix E.3 distinguishes the mathematical references from historical implementation behavior.

   Guidelines:

   - The answer [N/A] means that paper does not include experiments requiring code.
   - Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
   - The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
   - The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
   - At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
   - Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: [Yes]

   Justification: The paper reports the data setting, training protocols, augmentation configurations, baselines, evaluation metrics, and optimizer-related details needed to interpret the results, with appendix tables providing the fixed hyperparameters and additional implementation details.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [Yes]

Justification: We report means and one sample standard deviation over three seeds for the ImageNet-1K, CIFAR-100, and cross-architecture comparisons, three seeds for ImageNet-LT, and five seeds (42–46) for the loader benchmark. COCO results summarize three pretraining seeds (42–44), each with one transfer run using transfer seed 42; the optimizer and hyperparameter sensitivity studies use single-seed runs and do not estimate variation across seeds.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [Yes]

   Justification: We explain the compute resources used for the experiments in the experiments section, including the relevant hardware setting for reproducing the reported runs.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

   Justification: We adhere to the NeurIPS Code of Ethics.

   Guidelines:

   - The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.

- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [N/A]

    Justification: We propose a general technique for image classification training, and its possible downstream uses are too broad and indirect to support a concrete societal-impact analysis within the scope of this paper.

    Guidelines:

    - The answer [N/A] means that there is no societal impact of the work performed.
    - If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
    - Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
    - The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
    - The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
    - If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

    Answer: [N/A]

    Justification: The paper introduces a general image-classification training method and does not release any high-risk models, scraped datasets, or other assets that would require special misuse safeguards.

    Guidelines:

    - The answer [N/A] means that the paper poses no such risks.
    - Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
    - Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
    - We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: We use existing open-source software libraries and standard public datasets, cite the original sources, and follow their respective licenses and terms of use. We do not use proprietary assets

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes]

Justification: We release the implementation of TreemapMix, including the augmentation itself and both proposed supervision variants.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: The paper does not involve crowdsourcing nor research with human subjects.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Justification: Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.

- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [N/A]

    Justification: The paper does not involve crowdsourcing nor research with human subjects.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
    - We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
    - For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

    Answer: [N/A]

    Justification: LLMs are not an important, original, or non-standard component of the core methods in this research.

    Guidelines: The core method development in this research does not involve LLMs as any important, original, or non-standard components.

    - The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
    - Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.